RELbot Software and Usage Manual (v2025.1)

Gijs van Oort, Luuk Lenders, Jan Broenink

Monday 26th January, 2026

Contents

1 Introduction **1**
1.1 Goal of this manual 1
1.2 Structure and use of this manual 1
1.3 Document History and Disclaimer 2

2 Software Development in General and for Robotics **3**
2.1 Software Development in General 3

3 Install C++ Integrated Development Environment **5**
3.1 Introduction 5
3.2 VS Code as IDE and g++ as C++ compiler 5
3.3 Install VS Code and g++ compiler 5
3.4 Testing g++ via compiling (building) from the command line 6
3.5 Configuration files in VS Code 7
3.6 Compiling / building and debugging using VSCode 7

4 Install for Remote Development **9**
4.1 Introduction 9
4.2 Install virtual-machine managing software and a Linux virtual machine 10
4.3 Install ROS2 Humble 13
4.4 SSH key: Arrange the connection between a host and a VM or RELbot 14
4.5 Configure VS Code for remote development, on a VM or RELbot 15
4.6 Start developing C++ programs on a remote site 16
4.7 Install VS-Code extensions for ROS2 17
4.8 Arrange showing graphics from a VM or RELbot on the host machine 17
4.9 Install ROS2 directly on Linux 18
4.10 Configure VS Code to build C++ programs on the Raspberry Pi 19

5 Structuring, Documenting, Building, and Handing in C++ Projects **21**
5.1 Introduction 21
5.2 Structuring C++ projects 21
5.3 Commenting and Coding style and support for it by VS Code 21
5.4 File Comment Header 22
5.5 Build / Compile C++ projects 23
5.6 Running and Debugging the generated program 25
5.7 Operating-System specific system calls 25
5.8 Testing a multithreaded program 25

5.9 Prepare for Hand-in . . . 25
5.10 How graders/TAs grade your C++ projects . . . 27

6 Structuring, Documenting, Building, and Handing in ROS2 projects 28
6.1 Introduction . . . 28
6.2 Structuring ROS2 projects . . . 28
6.3 Commenting and Coding Style of ROS2 nodes and support for it by VS Code . . . 29
6.4 Building ROS2 projects . . . 30
6.5 Running nodes . . . 31
6.6 Launch files . . . 31
6.7 Preparing a hand-in for ROS2 projects . . . 33
6.8 How graders/TAs grade your ROS2 projects . . . 34
6.9 Examples of README files . . . 34

7 Webcam stream to the VM 36

8 Accessing the Raspberry Pi of the RELbot 37
8.1 User name/account . . . 37
8.2 Connecting to the Raspberry Pi (SSH) . . . 37
8.3 Avoid loosing files . . . 37
8.4 Remarks and Tips . . . 38

9 XRF2, the Xenomai 4 – ROS 2 framework 39
9.1 Structure of the Framework . . . 40
9.2 Using the framework in your project . . . 40
9.3 Background Information on XRF2 . . . 43

10 Using the XenoFrtLogger 46

11 Software Development on Xenomai 4 / EVL 48
11.1 Introduction . . . 48
11.2 C++ code for Xenomai 4 / EVL . . . 48
11.3 Configuring VS Code to build Xenomai 4 code on the Raspberry Pi . . . 50

12 Tips & Tricks and Cheat sheets 52
12.1 Introduction . . . 52
12.2 General terminal commands . . . 52
12.3 VS Code . . . 52
12.4 Compiler (g++ or clang) . . . 52
12.5 WSL (Windows) . . . 53
12.6 Multipass (Mac) . . . 53
12.7 SSH . . . 53

12.8 ROS 2 53
12.9 Colcon 54
12.10 Terminal applications useful for ROS 54

13 RELbot **56**
13.1 Introduction 56
13.2 Overview of the RELbot 56
13.3 Software Overview 57
13.4 Hardware overview 58

14 RELbot Start up, Shut down, Battery, Store in Box **61**
14.1 Quick Guide 61
14.2 Start up 61
14.3 Using the RELbot 61
14.4 Shut down, store in box 61
14.5 Background on batteries and power supply 62

15 Developing Soft Real-Time Software for RELbot using ROS2 **63**
15.1 Introduction 63
15.2 RELbot_Simulator ROS2 Node 64
15.3 From coordinates to image 66
15.4 RELbot_adapter node 66
15.5 Keyboard ROS2 Node 68

16 Developing Firm Real-Time Software for the RELbot using 20-sim and XRF2 **70**
16.1 Introduction 70
16.2 RELbot Dynamic Model in 20-sim 71
16.3 Prepare 20-simsubmodels for use in XRF2 72
16.4 Background information on code generation using 20-sim 73

A Change Log **74**

Bibliography **75**

1 Introduction

1.1 Goal of this manual

This document contains instructions and tips for software development for robotics and testing of the resulting software for the RELbot – *Robotics Education Lab robot*. This document also contains instructions on how to start up, change batteries, shut down, etc the RELbot; The *Usage* part of the manual.

However, this document is also useful for developing software using VS Code as development tool and/or ROS2 for robotic/embedded systems containing a Raspberry Pi computer or comparable *not* part of a RELbot.

Software development is on constructing well-structured and well-documented code whereby the resulting code should be insightful for others who use it and for later use by yourself. Thus, the *design decisions* motivating the code are of utmost importance. Of course, the computer should execute the code to produce right results correctly and rather efficiently.

In case of software for robotics, *testing* is more complicated than for general software, because the robot movements influence the software via sensor values, as reaction on the steering of the robot by the control algorithms executed as software. This implies that the mechanical part of the robot, or a simulation of its behaviour is needed to be able to run a good test. We provide a RELbot simulator to organise testing and debugging in an effective manner.

See Chapter 2 for more on properly developing software in general and more specific for robotics.

1.2 Structure and use of this manual

An overview of the chapters is shown in Table 1.1. Software development and tools in general are in Chapters 2 to 7. Software developing for a Raspberry Pi prepared to control motors using our XRF2 framework is in Chapters 8 to 12. *Specifics* about the RELbot are in Chapters 13 to 16.

Table 1.1: Structure of this manual. A: for all; R: for advanced real-time programming; C: for development of firm real-time control laws and time-critical safety software.

Chapter	Use	Topic
2	A	Software Development approach
3	A	Installation of VS Code and C++ compiler
4	A	Installation of a Virtual Machine and remote environment
5	A	Structuring C++ projects, including rules for hand in
6	A	Structuring ROS2 projects, including rules for hand in
7	A	To get webcam streams into the Virtual Machine on your laptop
8	A	Access to Raspberry Pi
9-10	A	Use of our XRF2 real-time framework
11	R	Real-time software development in Xenomai 4 / EVL
12	A	Tips and Tricks of the tools
13	A	Intro to the RELbot, its hardware and software
14	A	Using the RELbot, its batteries, and remote connection
15	A	Soft real-time software: In ROS2 nodes and simulation using ROS2
16	C	Firm real-time software: Loop controllers etc in XRF2 via 20-sim simulation

Not all chapters are needed in all cases. This depends on the specific software development task to be done. Therefore, it is recommended to use this manual as a *reference*, so reading a specific chapter when needed and not reading everything linearly from begin to end.

Notes

- Chapters 1 and 2 should be read at the beginning of *all* software projects.
- Chapter 14 should be read before using the real RELbot for the first time.

1.3 Document History and Disclaimer

This document has grown from appendices for courses of software development towards the independent manual on software development for robotics as it is now.

When you find a mistake or have remarks about this document, please contact one of the authors.

As we also continuously improve this document, newer versions appear regularly, see its date and version number. Updates and changes are summarised in the Changelog, in Appendix A.

The latest version of this document to be used in coursework is on the Canvas site of those courses. It is also on the GIT server of RaM (in RoboticsEducationLab). Be sure to always use the latest version of this document.

This document has seen quite some versions over the years. Contributors to this document, next to the authors, are: Ilyas Raoudi, Jurrijn Pott, Jelle Hierck, Jasper Vinkenvleugel.

Development and documentation of the RELbot is by Gijs van Oort, Sander Smits, Marcel Schwirtz, Marjon Kuipers, Artur Gąsienica, and Jan Broenink.

2 Software Development in General and for Robotics

C++ is a very powerful and complex language and has many keywords and ways of structuring the code. The self-explanatory level of C++ is rather modest (for sure compared to other modern languages like *Python*). Therefore, clever use of C++ constructs, carefully structuring the software, and a consistent style of coding and giving names to functions and variables is really important.

2.1 Software Development in General

The importance of good software development is clear: we need to construct well-structured and well-documented code such that the resulting code is insightful for others who use it and for later use by ourselves. Thus, the *design decisions* motivating the code and proper *structuring* the code are of utmost importance. Of course, the computer should execute the code to produce right results correctly and rather efficiently.

Structuring the code and making that readable for others must be done explicitly by the software developer, as the compiler does not give warnings for violating agreements about this. These agreements are *not* part of the language (not part of any computer language). So, this structuring needs careful attention.

In the following four topics documenting and structuring C++ software is explained.

2.1.1 Be Explicit in Design Decisions

Explain the *why* of design choices of the *final* code in a *separate* document, including testing of the software:

- Why is the choice made for the existing:
  - Classes / objects
  - Relations between classes
  - Attributes (variables) in each class
  - Division of functionality in methods (functions)
  - Implementations of the methods
- Why have these tests been conducted? To make plausible that the software is right.
  - What is the structure of the testing and test bed?
  - What regular situations have been tested?
  - What special conditions have been tested?
  - What exceptional cases have been tested?
- Show the testing, as evidence that the software works right.
  - First describe the tests.
  - Then, show results in a compact way.
  - Draw conclusions from the test about correct functioning of the code. Clearly separate your interpretation of the results from the results themselves.

2.1.2 Carefully Structure the Code

Structure the code according to the usual style of object-oriented programming as is shown in Deitel and Deitel (2017):

- Use two files per class by separating declarations from implementation (*.hpp and *.cpp). In Deitel and Deitel (2017), this is only used from Chapter 9 onwards (10th edition).
- Make attributes (local variables) always private. Use get and set functions to retrieve and update the attributes. Check in the set functions the correctness of the values with which the attributes are to be updated.

- Structure the methods (code in the functions) according to the algorithm that the code implements. However, take into account the following:
  - For long pieces of code, split it into logical parts and thus into several functions.
  - In case classes have functions in common, push them up in the hierarchy, using inheritance. This to avoid duplicating code.
- Use a separate test program (*test bed*) to separate this code from the code of the algorithms to be tested (*Code Under Test* or *(Software) Component Under Test*, CUD). Both for generating inputs and starting the code of the algorithms, and for code to read and present the outcome of the running code, see Figure 2.1. In Deitel and Deitel (2017), this is called a *driver* program.

Figure 2.1: Structure of a *test bed*: the software *Component Under Test* is tested.

2.1.3 Consistent Coding Style

Use a consistent coding style, to support quicker understanding and easier reviewing of the code:

- Follow the coding style as used in Deitel and Deitel (2017).
- Use meaningful names for all identifiers/names. Each name must precisely indicate its role. Comply with naming conventions, namely *Camel* case, like setXxx, getXxx, xxxPtr.
- Use const variables to denote numbers. *Never* use literals (called *magic numbers* in software-engineering terminology). This for easier reuse, more robust code, and more support by the compiler.
- Use the standard file header comment in each file. See Section 5.4 for details and automating it using VS Code.
- Document each declaration. So, *one* declaration plus its comment per line.
- Document each method. Before: a general comment; inside: at relevant places to explain the algorithm.
- Put comments *always directly above* the code, or in case of declarations, on the *same* line.

Note that automated document-generation tools exists, like doxygen that generates documents from comments in source code using special comment blocks with some additional markings. Although it is quite easy to do (for sure when using its VS Code extension), we do *not* ask you to do so in the context of courses at UT. For larger projects like BSc or MSc-Thesis projects, it is a good idea to use doxygen-style comments.

2.1.4 Compute right things right

Of course the right things must be computed correctly, that is a correct result must be produced that complies with the requirements (assignment). Take into account the following:

- Compute the right things, so, what is asked.
- Compute the things right, so, produce correct results in all cases. Testing is important here. Use separate test programs (see above).
- Efficiency of the computation is interesting but not an important requirement. Elegance is more important, like motivated choice for recursion or iteration. Structure of code is more important, see above.

3 Install C++ Integrated Development Environment

3.1 Introduction

The big advantage of IDEs is that, next to editing and look-up support, a lot of details on how to compile and link are hidden behind a graphical interface. The downside is that it may be hard to understand what happens behind the scenes of the graphical interface. Furthermore, relying on the IDE keeps the compilation commands implicit, taking more work when you have to share / submit the code including those compilation instructions. Therefore, it is good to know how things work under the hood, especially compiling and linking from the command line.

Note that for submitting assignments on software-development course work, you have to produce README.txt files in which you describe how to compile and run each program from the command line (including the exact commands that you use). This to be sure that those who grade your work can easily compile and run your code.

About VS Code as the 'best' IDE and on installing it, is in Section 3.2 and Section 3.3 respectively. Installation of VS Code and g++ compiler and tuning VS Code is in the rest of this chapter.

More on compiling with g++, structuring C++ files and documenting C++ code, including producing README.txt files is in Chapter 5.

3.2 VS Code as IDE and g++ as C++ compiler

Although many IDEs exist, we recommend using VS Code as IDE as that is one of the mainstream IDEs and allows the TAs to give better support compared to using another IDE.

Furthermore, it is available for Windows, Mac, and Linux and has an enormous amount of available extensions. Both VS Code itself and its website provide documentation and help including tutorials.

Note that VS Code is *not* the same as Visual Studio. Visual Studio is another popular choice for IDE and is also made by the same developers as VS Code, but the TAs cannot support you with Visual Studio while developing software in Robotics courses.

The C++ compiler depends on the Operating System (OS) you have. It can actually be any compiler that is fully compliant with the C++14 or C++17 language definition. Fortunately, all mainstream C++ compilers comply with C++ standards, and you can tune for which standard code is checked / compiled. The code you hand in must be portable C++ code (that is, it will compile and run on any of the OSes used, most notably, those of the TA who checks your work).

3.3 Install VS Code and g++ compiler

VS Code is an editor which uses plugins ("Extensions") to connect with various compilers (not only C++; for almost every language extensions are available). VS Code runs on Windows, MacOS, and Linux.

For elegantly installing VS Code as IDE and the C++ compiler, we can let VS Code do the work. Therefore, start with installing VS Code, add its C++ extensions, and install C++ compiler according to the instructions given in those extensions and using the information below.

Install VS Code

Download VS Code from https://code.visualstudio.com/, and follow installation instructions given there.

On Linux (Ubuntu) you need the snap installer to install VS Code from the command line:
`sudo snap install --classic code`

Install VS Code Extensions for C++ and C++ compilers

Install the C++ compiler and VS Code extensions *C/C++*, *C/C++ Extension Pack*, and *C/C++ Themes*[1] via the documentation area of the VS Code website, by choosing C++ in the overview on the left-hand side and then selecting:

- For Windows: GCC on Windows.
- For Mac: Clang on MacOS.
- For native Linux: GCC on Linux.

If you like, you could check the Intro Videos via the first item of the C++ list of topics in the overview on the left-hand side.

We use the g++ compiler, and that is installed following the selections above. Do select the g++ compiler when compiling from within VS Code.

Follow the instructions about installing compilers at least up to and including "Run hello-world.cpp", such that VS Code and the g++ compiler are tested using the famous basic C++ program “Hello World”.

For the software development courses, you need to run the g++ compiler explicitly from the command line, so *not* from within VS Code. This makes the work of writing the `README.txt` file easier, and eases the grading work. This is treated in Chapter 5 together with tips and guidelines on structuring and documenting the code.

Tips and Remarks

- The information in the description of the C/C++ extension is also on the installation page mentioned above.
- For MacOS, only `Clang` is treated, but `g++` is installed in the same go using those instructions. On a Mac you can use both `clang++` and `g++`, as these commands both link to the clang compiler front end preparing for the same compiler.
- The *C/C++ extension pack* contains popular and useful extras for developing C/C++ code.
- When compiling using VS Code’s interface (as the installation manual suggests), be aware to select the g++ compiler, so *not* the gcc compiler, which is often the first to select from in the drop-down box of the command palette. The "++" is important here.
- In case compiling using VS Code’s interface gives errors, you might test the compiler using the *command line*, as treated in Section 3.4.
- For assignments of the software development courses, you need to run the g++ compiler explicitly, that is from the *command line*, so *not* from within VS Code. See Chapter 5 for this, next to tips and guidelines on structuring and documenting the code.
- The documentation part of the VS Code website contains a lot more documentation including videos. We have summarised some essential parts in Section 3.5 and further sections.

3.4 Testing g++ via compiling (building) from the command line

The default Command Prompt (on Windows a.k.a. Terminal) or Terminal (on Mac), being a separate program, suffices, although there are more versatile terminal programs available useful for more complex tasks, like developing and running ROS2 programs.

VS Code has a Terminal integrated in the *Panel*, the part below the editor region, which is already navigated to the folder of the project you are working on.

[1] Extension Pack and Themes are extra packages, and not mentioned in the documentation area of VS Code.

In the terminal go to the helloworld folder using the command cd. If you use the terminal integrated in VS Code, you are already in the helloworld directory.

Run the g++ compiler to compile and link the source code into an executable:
g++ helloworld.cpp -o helloworld

Test the program by running it (note the dot and (back)slash before the name of the program):
./helloworld on MacOS and Linux.
.\helloworld on Windows.

Remarks

- In Windows the extension .exe is automagically generated.
- Mac and Linux executables do not need a specific extension.
- More on compiling and linking is in Section 5.5.

3.5 Configuration files in VS Code

This section is here for completeness and further tuning of VS Code for the building / compilation and linking process. You would *not* need it when simply compiling C++ projects from the command line, or using the ROS build support when compiling ROS nodes (programs).

Configuration of VSCode usually goes per project/(VS Code-)workspace. When you open a folder in VS Code, it creates a folder .vscode in the base folder of the project, containing one or more settings files. These files are human-readable (and -editable) and stored in that .vscode folder:

settings.json	General project settings such as the colour scheme used.
tasks.json	Compiler/Build settings.
launch.json	Debugger settings.
c_cpp_properties.json	Settings for correctly parsing/checking C/C++ files.

For example, you supply include-paths here so that the VSCode can look up header files and use the information in there for code completion, real-time error checking, hinting etc.

You can edit these files so that they optimise the integration for your system. For example, they can contain paths to header files that VSCode than can use for syntax highlighting and code-completion. Some VSCode extensions provide (some of) the configuration files automatically.

Note: Some settings are stored in settings files for *all* projects, reachable via the menu entry Settings...

settings.json	To store generic settings, reachable via Settings.
cpp.json	To store *snippets* of cpp code, reachable via Configure Snippets.

3.6 Compiling / building and debugging using VSCode

This section is here for completeness and further tuning of VS Code for the building / compilation and linking process. You would *not* need it when simply compiling C++ projects from the command line, or using the ROS build support when compiling ROS nodes (programs).

3.6.1 Compiling / building

Building a project is started via Ctrl-Shift-B (Windows) or Shift-Cmd-b (MacOS), see also the *Terminal* menu. Building instructions for VSCode are in the .vscode/tasks.json file. If there is no such file, a default building task can be chosen from a popup menu. Choose the building task with the recommended compiler for your OS (g++). You might need to specify the language standard, for instance "–std=gnu++14" or "-std=c++17".

Note that the default building task *only* compiles the active file.

For a multiple-C++-file project, you need to customise the build task. Make sure you have .cpp as active (editor) file. Open the Command Palette (Ctrl-Shift-P or Shift-Cmd-p), and type Tasks: Configure default build task. Choose the appropriate build task that serves as your starting point. This creates a .vscode/tasks.json file which you can edit. Some things which you may want to change as stated below.

The file that is compiled is ${file}. You can modify this to, for example, "${workspaceFolder}/*.cpp" or "${workspaceFolder}/src/*.cpp" to compile and link all cpp files that are in the VSCode base folder or the src folder under the VSCode base folder respectively. Since the tasks.json file is specific for your current project, it makes sense to specialise it and explicitly list all cpp files that need to be compiled. In that case, each file is a separate entry, e.g., "${workspaceFolder}/src/main.cpp", "${workspaceFolder}/src/divide.cpp".

The name of the generated executable is the line after the "–o" argument; its default is "${fileDirname}/${fileBasenameNoExtension}". This implies that as soon as your active file changes (i.e., you start editing a different file), the executable name changes (because ${fileBasenameNoExtension} changes value). This is often not what you want. Instead, you can supply a fixed file name, e.g., "${workspaceFolder}/divide.exe" (or without the .exe part for Linux/MacOS).

If you have problems with getting your header files included, specify the folder where the include files are using the –I flag.

3.6.2 Debugging

Start *debugging* a project by F5, see also the *Run* menu. Debug instructions for VS Code are in the .vscode/launch.json file. If there is no such file, a default debugging configuration can be chosen which only debugs the active file. Prior to debugging the file is (re)built as well.

Debugging a multiple-C++-file project is handled automagically. If you have configured the building task correctly, the default debugging task is chosen accordingly. No need to change things here.

If you want more control on the debugging process, you can customise .vscode/launch.json. From the pull-down menu, choose “Run|Add configuration”. An ‘empty’ .vscode/launch.json file is created. In the edit window, click the ‘add configuration’ button and choose ‘C/C++ (gdb) Launch’. This gives you a starting point for your customisation.

Debugging only starts to become fun if you add breakpoints. To add/remove a breakpoint at a certain line, put the cursor on the line and press F9. A red dot appears in front of the line. Alternatively, click on the place where the red dot is about to appear.

3.6.3 Troubleshooting

If you think you may have messed up your compile/build/debug configuration, you can remove the .vscode folder in your base folder to start from scratch again (actually, just removing tasks.json and launch.json suffices; the other configuration files do not have anything to do with building/debugging).

There are also extensions that claim to do this configuration work for you, but our experience is that it is still good to know what is going on under the hood, such that you can fix any inconsistencies that the extensions might introduce.

4 Install for Remote Development

4.1 Introduction

The *remote development environment* presented in this chapter is for development on a *virtual machine* (VM) and for real remote development, like on a RELbot or a Raspberry Pi. So it is about the middle and right leg in Figure 4.1, being a recall of **??**.

We do *not* go for a complete virtual machine like those in Virtual Box, UTM, or Parallels, as this approach takes more resources from your laptop. Furthermore, in our case you can use your own tools running on your own laptop to do code editing, file handling etc., so *no* need to completely dig into the Linux environment. The advantage of using a VM is that that part is rather isolated from the rest: no contamination of the host system itself, and when facing issues, simply deleting the VM and install it again cures a lot.

By first developing using ROS2 on a virtual machine on your own laptop and thereafter continuing the development on the RELbot, you use the *same* development environment. This makes testing and real execution more or less the same in both cases from the point of view of the development environment. Furthermore, it makes you less dependent on *always* needing a RELbot available.

Note: The installation work differs per host computer (Mac or Windows) and per target (Virtual Machine or RELbot / Raspberry Pi), so *not* all parts of this chapter are applicable to all situations. However, quite some steps are the same, see Figure 4.2.

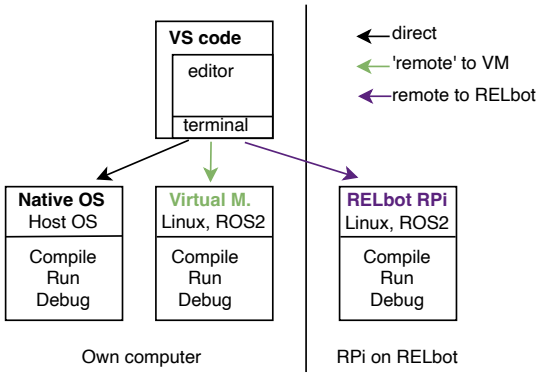

Figure 4.1: Software development workflow. Left: local; Middle: 'remote' using a VM; Right: remote. This figure is a recall of **??**.

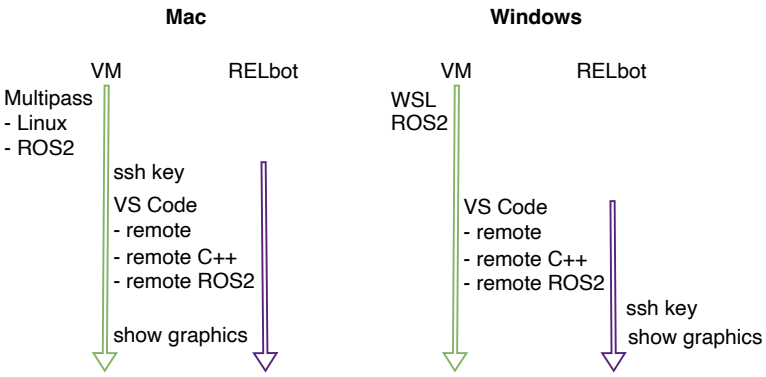

Figure 4.2: Different sequences of installation steps. Steps on both sides of the install-flow arrow must be taken to install the complete set of tools for that particular host / target combination.

The installation work consists of the following steps:

- Install virtual-machine managing software (different for Mac and Windows).
- Install Linux, version Ubuntu 22.04, *Jammy Jellyfish*, the "Long-Term Stable" version that runs with our firmware on the RELbot.
- Set up a mount point to reach your data files stored on your host OS.
- Install ROS2 Humble, the "Long-Term Stable" version for Ubuntu 22.04.
- Arrange the connection between the host machine and a VM or RELbot.
- Arrange the connection between VS Code and a VM or RELbot.
- Setup remote C++ development on a VM or RELbot.
- Arrange showing graphics from the VM or RELbot on the host machine.

These steps are explained in the next sections. Of course, tests for each step are indicated.

More information, including some background, is in the VS Code Remote Development section of the VS Code documentation, although this documentation is sometimes a bit too much inclined towards Windows (VS Code is developed and owned by Microsoft). This chapter is not, obviously.

4.2 Install virtual-machine managing software and a Linux virtual machine

Mac and Windows need a different approach: On a Mac we use Multipass and for Windows, we use WSL (Windows Subsystem for Linux) as virtual-machine managing software.

Before this install work, first terminal programs (Command prompt) with multi-pane capabilities are explained, as such a facility is really useful while developing and running ROS2 programs.

4.2.1 Advanced Terminal Programs

The default Command Prompt (on Windows a.k.a. Terminal) or Terminal (on Mac), being a separate program, suffices, but becomes unhandy when many terminals need to be open to control and run all your ROS2 programs / nodes.

Terminal programs like *PowerShell* or *Windows Terminal* (on Windows) or *iTerm* (on Mac) have quite some configuration possibilities, especially for the remote and ROS node/program development we are doing, namely several terminal panes next to each other in one application window.

VS Code has a Terminal integrated in the *Panel*, the part below the editor region, which is already navigated to the folder of the project you are working on. It has a facility to create more terminal panes, also next to each other.

Building and running ROS2 nodes/programs is often more convenient from an (advanced) terminal program than from the terminal pane in VS Code. However, it is a matter of taste.

Consider installing an advanced terminal program first, before doing the installation work. It is not necessary, though. So, you can safely install it later.

4.2.2 Mac: Install Multipass, Linux, and also ROS2

Multipass has a well-structured set of documentation in their documentation area, on their website, menu entry “Docs”. For a quick start, some relevant commands are listed on the install page.

Download and install multipass from its website. It must be at least version 1.16.1+mac.

Start Multipass.

Linux (Ubuntu) and ROS2 can be installed via two different ways, where in both cases tuning the Virtual Machine (*instance* in Multipass speak) can be done via the Multpass GUI:

- Ubuntu via the Multipass GUI and ROS2 via command line in the virtual machine.
- Ubuntu and ROS2 in one go via the command line on the host machine.

Installing Ubuntu via the Multipass GUI, by selecting in the Catalogue *Ubuntu 22.04 LTS* and tune its parameters, see below.

Installing Ubuntu and ROS2 via the command line using *cloud-init*, saving some typing:

- In the *How-to guides* of the documentation, go to *Manage instances* and select *Launch customized instances with Multipass and cloud-init*, scroll down to *ros2-humble*.
- Copy the indicated commands in a terminal and execute that command. Parameters of this instance can be updated before executing the command or in the Multipass GUI, see below.
- You need only to configure the VM and test ROS, see Secions 4.3.2 and 4.3.3, so you can skip the rest of Section 4.3. You need to update permissions of the .bashrc file, see Section 4.3.4.

Tuning parameters of the Multipass *instance* via the Details tab of the instance in the Multipass GUI:

- Name of VM: choose *ros2-humble*, the name of the ROS2 version we are using, or any other suitable name.
  Note: the name of the VM (instance) can only be specified when it is to be created (launched).
- CPUs: not more than half of the available cores, 4 will do anyway.
- Memory: not more than half of the available memory, 8 GB will do.
- Disk: 40 GB is recommended.
  This 40 GB is not immediately taken, fractions are only claimed when needed.
- Bridged Network is not needed.
- Mounts: No need to change, so keep your Mac home directory (host directory) mounted in the Ubuntu-user directory.
- After the VM and ROS2 are set up, add an extra mount point to the src directory of the ROS workspace. So:
  - Host (=Mac) directory: /Users/<your Mac-Homedir>/<...>/<Mac-ros_ws>/src
  - Guest (=VM) directory: /home/ubuntu/ros_ws/src

  You need to first create the directories on your Mac, and the directory ros_ws in your home directory on the VM. This is needed to let colcon, ROS2's build system, to work correctly, as the directories generated by colcon must be in the VM's file system.

We advise using a mount point, such that your files can stay on the file system of your host OS, and are available in the VM. Using such a *mount* keeps your data (uh... programs) separate from the Ubuntu / ROS2 environment, so you would not lose your data and programs in case the VM gets corrupt. This way of working is often done in industry.

You can work in Ubuntu / ROS2 from the Multipass graphical user interface or directly from a macOS Terminal (a.k.a. *command prompt* or *Shell*) using the command:

```
multipass shell ros2-humble
```

Note: the Shell from within the Multipass GUI might have some restrictions w.r.t. use of the clipboard. So, using a MacOS Terminal as Shell might be easier.

Start a *shell* (Terminal in Mac speak) from either within the Multipass user interface or from a macOS Terminal and note the IPv4 IP address, as that is needed later to connect to this VM from the host OS. It is part of the internal local network between the host and the VM. This also shows that installation of Multipass and Ubuntu has succeeded.

4.2.3 Windows: Install WSL and Linux

WSL, Windows Subsystem for Linux, version 2, supports quite some Linux commands and thus Linux programs.

To install WSL, we follow the guide on the install page, but have defined some extra steps to make sure everybody is using the right version.

- Start a Command Prompt (or Terminal) and "Run as Administrator" by right clicking and selecting this option.
- Run the command wsl –– install ––no–distribution
  If WSL was installed before, it is indicated, most probably as "The operation completed successfully."
- To be sure to have the latest version (obviously skip this when you installed WSL in the previous step) by running wsl ––update
- Run the command wsl ––set–default–version 2 in your terminal to ensure using WSL 2.
- Run wsl –install -d Ubuntu-22.04 to install the right version of Ubuntu for this course.
- Choose a username and password (not always asked for). Choose as user ubuntu and as password ubuntu This Linux username has no relation with your Windows username.
- Reboot your laptop, as usual on Windows after these kind of installation work.

Some useful commands on handling Linux distributions on WSL are listed on the install page.

Note: for some very old Windows 10 versions, installing WSL take more steps, as explained on the install pages. Please follow those instructions if your system experiences this.

Some information about the VM is presented, including its IP addresses. Note the IPv4 address, as that is needed later to connect to this VM from the host OS when using graphics. It is part of the internal local network between the host and the VM.

Tips and best practices for a Linux VM running on WSL

- Starting the VM (Linux / Ubuntu using WSL) can be done via several ways:
  - Starting a *Shell* (Command Prompt in Windows speak) on the VM by just starting Ubuntu 22.04 as a regular app on your Windows machine. In that Shell window, navigate to the Linux home directory using the command cd ~
  - Starting the VM from a Command Prompt or Power Shell using the command wsl. After that, navigate to the Linux home directory using the command cd ~
  - Starting the VM, and navigating to the Linux home directory using the command wsl ~
- In Linux, your home directory is /home/ubuntu, which is also where command cd ~ in the ubuntu environment will bring you. It is recommended to put any files starting from this location. This includes your ROS2 files. Starting a VM in a shell puts you at the location you called the command from, so you must do an explicit navigation to the Linux home directory, with the indicated cd command. See the *File storage* Section on the *Best practices for setup* page of the *Tutorials* part for more info on file structure.
- Some status info including the hostname and IP address is shown only *once* per day. The IPv4 address can be queried via wsl.exe hostname –I inside the WSL environment. You can also request this info in a normal Windows shell by using wsl hostname –I
- File Explorer can be started from a VM (linux) Shell using explorer.exe .
  Note the dot at the end of this command. This indicates to let the Explorer show the contents of the current directory.
- *Ubuntu 22.04* in the file path at the beginning or not, reveals whether a directory on the VM is shown or on 'normal' Windows.
- Windows programs can be started from within a Linux Shell by using the extension .exe

- When using either PowerShell or Command Prompt, turn on Allow copy / paste via the Clipboard to Windows by enabling this feature in the settings. Reach these options via right click in the top bar of the shell and choose options. This might be called Enable ctrl+shift+C/V.
- You could use the program wslsettings to check and update settings of your virtual machines. However, default values are OK for our work.
- The Tutorials section of the online WSL documentation contains the page Best practices for set up, with a few good tips concerning getting this VM running.

4.3 Install ROS2 Humble

Mac and Windows have the *same* installation process for ROS2 Humble, obviously, as ROS2 is installed in the (virtual) Linux environment. Even for a Linux machine the installation process of ROS2 Humble is the same.

4.3.1 Installation steps

Installation instructions are on the ROS2 site.

Note that we do *not* use the newest version of ROS2. However, we need Humble, because of the combination with the firmware running on the RELbot.

You can obviously skip this part when you have installed Ubuntu and ROS2 in one go (macOS only). However, you need to do the configure and test steps in Sections 4.3.2 and 4.3.3.

Do use the recommended approach for Ubuntu Linux (*deb packages*), as that installation is nicely integrated in the Ubuntu we have. Do install the optional development tools, so ROS2 package ros–dev–tools. You will be developing ROS2 nodes and packages.

Desktop installation gives the complete set of tools and packages you need, so install ros-humble-desktop. This installation includes demos, tutorials, and graphical tools. We do *not* use all graphical tools, as we use the graphics on the host OS, but this desktop version is really useful to have.

In case you have not that much disk space, you can opt for the *ROS-Base* install, and install material for demos and tutorials when needed (at the tutorials, it is indicated what to install).

4.3.2 Configure the Linux VM for ROS2

Add the location of ROS2 to the settings file of the Ubuntu Shell (.bashrc), to avoid setting this every time you start a new Shell (Terminal), by typing
echo source /opt/humble/setup.bash » .bashrc
while being in your home directory, /home/ubuntu. Change directory to your Ubuntu home directory is via the command cd ~

Mac: add a mount point to the src directory of the ROS2 workspace, see the *Tuning parameters* part at the end of Section 4.2.2.

4.3.3 Test ROS2

To test ROS2, run the talker - listener demo, as presented as the last step on the install page. Just run the talker and the listener, and do *not* change the *ROS Middleware* as is discussed on the install page:

- In one terminal run the C++ talker: ros2 run demo_nodes_cpp talker
- In a second terminal run the Phython listener: ros2 run demo_nodes_py listener
- **Note**: It is assumed that the VM is configured to run ROS2, as presented above.

You should see the talker and listener showing sentences about publishing and reading messages.

Before testing ROS2 programs that show graphics, first showing graphics must be arranged, see Section 4.8. A test is presented in that section.

4.3.4 Tips and Notes

- The command sets in the light green boxes can be copied to your terminal using the copy symbol in the top right, visible when you hover in that corner.
- Multi-line command sets starting with sudo might need some extra care, as the enter included at the end of a sudo command line can be interpreted as submitting an empty password, causing that (set of) commands go wrong.
- Most command sets can be run multiple times, except installation of *curl* in the box of *Now add the ROS 2 GPG key with apt.* Omitting the *-y* parameter on the first line, allows it do run more often.
- Mac: when installing Ubuntu and ROS2 in one go, you have to update the permissions of the .bashrc file first (seems like a bug) via the command sudo chown ubuntu:ubuntu .bashrc
- Windows: starting a Shell (Terminal) on the Linux VM puts you in the directory where you started this terminal. So, you have to first change directory to the home of the Linux filesystem (/home/ubuntu) via cd ~
- In case you installed the base version of ROS, you need to install the demo nodes before you can run the talker - listener demo, via the two commands:
  sudo apt install ros-humble-demo-nodes-cpp
  sudo apt install ros-humble-demo-nodes-py
- If you encounter the message *deferred to phasing*, packages / libraries are not immediately installed for reasons of quality / safety, as these parts have been reported on malfunctioning and are in the repair process by the Ubuntu distributors. As it is about updates and the current version is still there, the software still keeps running. So, you can safely ignore this message.

4.4 SSH key: Arrange the connection between a host and a VM or RELbot

4.4.1 When an SSH key is needed

On *Windows* using a VM via WSL, a secure connection is implicitly arranged, so *no extra* installation steps are needed for this situation (middle leg of Figure 4.1). So, *only* for this situation (host to VM) you can skip this section. To connect to the RELbot, you will need this section, see below.

On *Mac* using a VM via Multipass (middle leg of Figure 4.1), a secure connection via an *ssh key* is needed, see below, Section 4.4.2.

To connect to a *RELbot / Raspberry Pi* (right leg of Figure 4.1 and purple arrows in Figure 4.2), for both Windows and Mac, a secure connection via an *ssh key* is needed, see below, Section 4.4.2.

The connection between VS Code and the virtual environment respectively RELbot must be secure as must be for every remote connection. This is realised via a *Secure SHell*, *ssh*, taking care of the authentication when connecting to the remote side, see Section 4.5.

4.4.2 Set up a secure connection

Between VS Code and the VM from a Mac or between VS Code and a RELbot for both Mac and Windows, the secure connection using *ssh* must be configured explicitly before you can connect to that VM resp. RELbot. This holds also for a connection through VS Code using its extension *Remote - SSH*.

Establish the secure connection via ssh using key-based authentication, as supported by ssh, by putting the public-key of the private-public authentication key pair to the remote side. If you do not have such a private-public authentication key pair, you have to generate it.

Create an ssh key, if needed

Check if you have an ssh key on your *host* system by seeing if ~/.ssh/*<keyName>*.pub exists [1], where *<keyName>* is the name of the key as was generated, often (and in our case) id_rsa.

If there is no key present, generate one by running the following command in a terminal on the host (Mac or Windows):
ssh–keygen –t rsa
You can safely skip using a passphrase by hitting enter when asked.

This keygen program produces a public and private key in ~/.ssh and you can copy the public key to the VM (in case of a Mac) or a RELbot, as stated below.

Mac: Configuring a secure shell connection to the VM

Put the public part of the private-public authentication key on the remote side, the VM (Mac only), by copying the public key from the host to the remote side, by the following command, using a shell (terminal) on the host:
multipass exec ros2-humble −− bash −c "echo ‘cat ~/.ssh/<keyName>.pub‘ >> ~/.ssh/authorized_keys"
all on *one* line, where <keyName> is the name of the public key file and *ros2-humble* is the name of the Ubuntu VM (*instance* in Multipass speak).
Note: The copy-pasting from the pdf might not work, as the backquotes of the echo text appear as single quote, causing the command not to work. Mind the space between −− and bash. You could copy the ssh key from the host and add it in the authorized_key on the VM.

Mac: Connect to a VM

Simply using the *ssh* command and typing the password will do (assuming ubuntu is the user name): ssh ubuntu@<IPv4 address>

Note that you can also start a shell from within the GUI of Multipass. However, showing graphical output is then not possible.

Mac and Windows: Connect to a RELbot

Simply using the *ssh* command and typing the password will do:
ssh <user name>@<IPv4 address>

To prevent typing the password every time a connection is started, copy the public part of the authentication key to the RELbot in the file .ssh/authorized_keys:

- Mac: run the ssh–copy–id command instead of the normal ssh command from a shell (terminal) on the host while connecting to the Raspberry Pi.
- Windows: run the remote secure copy command:
  scp .*<keyName>*.pub *<user name>*@*<RELbot IPv4 address>*:./.ssh/authorized|keys

4.5 Configure VS Code for remote development, on a VM or RELbot

Mac and Windows are slightly different here. VS Code to the RELbot (its Raspberry Pi) is the same for both types of hosts.

[1] ~ is the shorthand for your home directory. On Mac or Linux is is usually /Users/*<loginName>* or /home/*<loginName>*. On Windows it is *<Drive Letter>*:\Users*<loginName>*

For developing C++ (or code in another language) some specific software on the remote site is needed. This is automagically taken care of when starting to develop code on the remote site, see Section 4.6.

For using ROS2, the *Robot Developer Extension for ROS2* by Ranch Hand Robotics LLC implements support for development for ROS2, see Section 4.7.

Install remote-SSH, for both Mac and Windows

On *Windows*, VS Code needs the WSL extension. Simply install the extension when a request to do so pops up.

On *Windows* and *Mac*, install the *Remote - SSH* extension of VS Code for support for secure shell connections to remote (virtual) machines.

Connect to a remote machine within VS Code

To start the remote connection press the ⊰-icon at the lower left of the VS-Code window. Choose

- Mac or Linux VM: *Connect to Host...* and specify *<user name>@<remote IPv4 address>*. User name is ubuntu and the IPv4 address as shown when starting a shell on the VM.
- Windows: *Connect to WSL*. Nothing to add.
- RELbot: *Connect to Host...* and specify *<user name>@<remote IPv4 address>*. Username is provided for each group, see Chapter 8, and the IPv4 address is shown on the display of the RELbot (labeled *eth0* for cabled access, and *wlan0* for WiFi access, both only via a dedicated VPN. In the RaM-lab, the VPN is called RaM-lab.

While connecting for the first time, VS Code automagically installs some 'server code' at the remote site to support this connection. It puts even more on it when starting with developing software, see the next subsection.

4.6 Start developing C++ programs on a remote site

While developing C++ code for the first time, VS Code asks to install the *C/C++ Extension Pack* extension at the remote side. Of course install this extension.

Windows: Install build tools on WSL

Install C++ compiler on WSL in VS Code, via the documentation area of the VS Code website, choosing C++ and then GCC on Windows Subsystem for Linux.

Follow the instructions about installing the compiler, debugger, VS Code extension and do the "Hello World" example. Explore a bit using the debugger and maybe intellisense.

Mac: Check your VM development system

The compilers are available on the Linux / Ubuntu VM, and can simply be used.

Start a remote connection to the VM via VS Code, and construct the "Hello World" project as you did while installing C++ compilers using VS Code on your host.

Some specific software on the remote site is automagically installed when starting to develop code on the remote site.

Notes and Tips – Mac

- Create this project in a *different* folder than the "Hello World" project on you host. You can safely store these project files on your host system, as these are available on your VM via the mount point point, as set up during configuring of the VM in Multipass.

- You can put your Ubuntu C++ files and projects in a folder tree next to the C++ files and projects that are compiled for your Mac.

Mac and Windows: Remote development on RELbot / Raspberry pi

The compilers are available on the RELbot / Raspberry Pi, and can simply be used.

Start a remote connection to the RELbot / Raspberry Pi via VS Code, and construct the "Hello World" project as you did while installing C++ compilers using VS Code on your host.

Some specific software on the RELbot / Raspberry Pi is automagically installed when starting to develop code over there.

Notes and Tips

- Create a folder (as usual) on the RELbot /Raspberry Pi to put your project in. Putting files on the RELbot / Raspberry Pi is needed as the programs you create must run over there.
- Do copy the files to your host after finishing your development. If you are familiar with file synchronisation tools or code development tools (like Git) you could use these to automate this file synchronisation / copying process.
- The debugger, `gdb`, might need to be installed. To do so, use the apt install command: `sudo apt install gdb`
- In order to let *Intellisense* to work for both the host and remote sides, you might need to use different VS Code *profiles*.
- For configuring VS Code to use its build support, see Section 3.5. However, for course work at UT this is *not* needed.

4.7 Install VS-Code extensions for ROS2

Install the extension *Robot Developer Extensions for ROS 2* by Ranch Hand Robotics LLC. It is the follow-up of the earlier-used and now deprecated ROS extension by Microsoft.

A key activity of this ROS extension of VS Code is to streamline developing and debugging of ROS2 nodes. Simply connecting to a remote ROS2 environment can be done by using an ssh connection, either from a separate Terminal / Shell, or from the Terminal window in VS Code.

See the documentation of this extension about its capabilities. Especially for issuing ROS2 commands via the *Command Palette*.

Note: The ROS extension of VS Code is normally installed at the remote site (VM or RELbot / Raspberry Pi) where ROS2 is installed. VS Code shows only extensions of the remote site when connected to that remote site.

Test the very basics by running some ROS2 nodes, like the talker - listener example used for installing ROS2 (see Section 4.3). Start these nodes in two different Terminal windows from within VS Code (use + in menu bar at the right-hand side of the Terminal pane).

See tips for attaching to a node etc via this extension, the documentation of this extension, reachable via the Extensions list in the Primary Side Bar (at the left-hand side of the VS Code window).

4.8 Arrange showing graphics from a VM or RELbot on the host machine

Showing graphics on the host is different for Windows and Mac with respect to preparation.

Using the VM on Windows via WSL does not need any extras for this.

4.8.1 Mac: Showing graphics from a VM or RELbot on your host – Preparation

For showing graphics (time plots of signals, for instance), we use an X server running on the host system. Such an X server is *not* part of the standard distribution of MacOS anymore. Fortunately, Apple has an X server available, namely Xquartz.

So, for Mac, simply install Xquartz.

Test it by simply using it, as presented in Section 4.8.3.

4.8.2 Windows: Showing graphics from a RELbot – Preparation

For showing graphics (time plots of signals, for instance) produced on the RELbot and to be shown on your Windows host, we use an X server running on the host system. We recommend X server from Xquartz. After you have installed X server, run the following installation steps:

1. Start XLaunch, you can search for it in the start menu.
2. You can keep the default settings, but you *must* enable the Disable access control. In case your firewall gives a warning, do allow vcXserv to communicate over networks (default value will do), of course.

Test it by simply using it, as presented below, Section 4.8.3.

4.8.3 Showing graphics from remote on your host – Usage and Testing

Mac users need to use ssh –X ... in both cases (from VM and from RELbot).

Windows users only need to use this ssh –X ... command to show graphics from the RELbot on the host.

1. Connect via ssh -X
   - Windows: wsl ssh -X <username>@<IPv4 address RELbot>
   - Mac / Linux: ssh -X <username>@<IPv4 address RELbot>
   - Mac: ssh –X <username>@<IPv4 address Multipass VM>
2. Testing the graphical connection
   Run turtlesim, a simple simulator provided by the ROS developers that is also used in RELbot software development.

Tips and Remarks

- Only the turtlesim_node needs to be run via an ssh -X connection.
- It is convenient to run all ssh connections as ssh -X. This to avoid confusion by accidently running a graphical program via ssh without graphical facilities.
- This -X argument / facility is *not* supported in shells started from within the graphical user interface of Multipass.

4.9 Install ROS2 directly on Linux

Installation instructions are presented in Section 4.3.

For Ubuntu users, there is a convenient script that does the full installation unattended.

Use the following three commands:
wget https://raw.githubusercontent.com/Tiryoh/ ros2_setup_scripts_ubuntu/main/ros2-humble-desktop-main.sh
all on *one* line.
chmod 755 ros2-humble-desktop-main.sh
./ros2-jazzy-desktop-main.sh

4.10 Configure VS Code to build C++ programs on the Raspberry Pi

This section is here for completeness and further tuning of VS Code for the building / compilation and linking process.

You do *not* need this when simply compiling C++ projects from the command line.

Use the Remote-SSH extension to login onto the Raspberry Pi. Make sure you have installed the C/C++ Extension Pack, both locally and on the Raspberry Pi.

The example tasks.json, launch.json and c_cpp_properties.json (Listings 4.1, 4.2, and 4.3) should get you started. When using these, you can press F5 to build and debug the C-file you are currently editing.

Notes and tips

- You can also use the same way of compiling C/C++ code that are described in Section 5.5.
- For UT course work, you *must* use compiling from command line, as the TAs need to also compile your code using a command-line command specified by you in the README file. Therefore, *only* relying on this task automation in VS Code is not enough.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C++: gcc build active file",
      "command": "/usr/bin/g++",
      "args": [
        "-fdiagnostics-color=always",
        "-g",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}",
        "-lpthread","-lrt"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": [
        "$gcc"
      ],
      "group": "build",
      "detail": "compiler: /usr/bin/gcc"
    }
  ]
}
```

Listing 4.1: tasks.json file for remote C++ development on Raspberry Pi

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "gcc - Build and debug active file",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "gdb",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ],
      "preLaunchTask": "C/C++: gcc build active file",
      "miDebuggerPath": "/usr/bin/gdb"
    }
  ]
}
```

Listing 4.2: launch.json file for remote C++ debugging on Raspberry Pi

```
{
  "configurations": [
    {
      "name": "Linux",
      "includePath": [
        "${workspaceFolder}/**"
      ],
      "defines": [],
      "compilerPath": "/usr/bin/gcc",
      "cStandard": "gnu99",
      "cppStandard": "c++14",
      "intelliSenseMode": "linux-gcc-arm"
    }
  ],
  "version": 4
}
```

Listing 4.3: c_cpp_properties.json file for remote C++ development on Raspberry Pi

5 Structuring, Documenting, Building, and Handing in C++ Projects

5.1 Introduction

The motivation of well structuring files and code, and using a good documenting style in a consistent way is explained in Chapter 2, especially Section 2.1.

On structuring files is in Section 5.2.

Quite some styles for documenting of C++ exist. It is important to select a good one and keep to that standard, that is, always apply it consistently. As indicated in Chapter 2, we use the style of Deitel and Deitel (2017), and have added to that a few rules on writing comments.

VS Code supports quite some coding styles, see Section 5.3 how to tune and use this. The standardised file comment header is specified in Section 5.4, including tips for automation in VS Code.

Building / compiling C++ projects is treated in Section 5.5. Running and debugging is in Section 5.6. A few operating-system specifics are in Section 5.7.

Basics on multithreading and testing such a program is in Section 5.8.

The rules to hand in C++ projects are in Section 5.9. How TAs grade your work is in Section 5.10.

5.2 Structuring C++ projects

For any not-so-small C/C++ project, *Good Programming Practice* is to split the source code into separate folders for *header files* (folder include) and *implementation files* (folder src). The header files (*.h / *.hpp) contain the declarations that are implemented in the source files (*.cpp). For *small* projects, both header files and source files can be in one project folder.

Other files (if any) should go to other folders. Graphics or data in folder resources. Files containing logs of the testing work in folder log. The README file explaining on how to use the software should be on the top level. See Figure 5.1a for an example.

This structuring adheres to the common approach as used in C/C++ software development. Reasons to do so is that often libraries are produced and published. In such a case the src/ folder content is *not* shared, as the code goes into the object files. The header files are needed though, to inform the compiler (and also developers who use the library) what classes and methods reside in the libraries.

For your assignments, each of the subassignments must be a new C++ project. This is done to make it easy for the TAs to assess and provide useful feedback for all of your code. It also makes it clear how to compile each of the subassignments. Most subassignments are standalone, i.e. you create new code from scratch. In case a subassignment is continuing work of a previous subassignment, you need to copy all code files from that subassignment to a new folder and continue your work there.

The folder tree in Figure 5.1a shows an example of how to structure your files for an assignment. Note that each subassignment's structure is again a structure of a C++ project.

On handing in a C++ project, including writing a README file is in Section 5.9.

5.3 Commenting and Coding style and support for it by VS Code

Add comments to the C++ classes in a structured way as stated in Section 2.1.3, so:

- Comments per file, using the standard file comment header, see Section 5.4.

```
assignmentX/
├── assignmentX.1/
│   ├── include/
│   │   └── MyCoolClass.hpp
│   ├── src/
│   │   ├── main.cpp
│   │   └── MyCoolClass.cpp
│   ├── assignmentX.1.exe
│   ├── assignmentX.1.o
│   └── README.txt
└── assignmentX.2/
    ├── include/
    │   ├── MyCoolClass.hpp
    │   └── OtherCoolClass.hpp
    ├── resources/
    │   └── pacman.bmp
    ├── src/
    │   ├── main.cpp
    │   ├── MyCoolClass.cpp
    │   └── OtherCoolClass.cpp
    ├── assignmentX.2.exe
    ├── assignmentX.2.o
    └── README.txt
```

(a) Project contents for two subassignments, where subassignment X.2 is a continuation of X.1 but the original implementation of X.1 is still stored for assessment and feedback.

```
Prog2-01-AssignmentX.zip
├── assignmentX.1/
│   ├── include/
│   │   └── MyCoolClass.hpp
│   ├── src/
│   │   ├── main.cpp
│   │   └── MyCoolClass.cpp
│   └── README.txt
└── assignmentX.2/
    ├── include/
    │   ├── MyCoolClass.hpp
    │   └── OtherCoolClass.hpp
    ├── resources/
    │   └── pacman.bmp
    ├── src/
    │   ├── main.cpp
    │   ├── MyCoolClass.cpp
    │   └── OtherCoolClass.cpp
    └── README.txt
```

(b) Hand-in zip file contents. The executable and object files are removed. Only the source files, code explanation and any resources (e.g. images) are preserved.

Figure 5.1: Example file structures for regular C++ projects.

- Comment each declaration, so *one* declaration per line, followed by its comment.
- Comment per method, directly above it.
- Add comments inside a method in case that piece of code is quite long or has some difficult coding constructs. Put it directly above the code the comment is about.

For developing C++ projects, the Deitel and Deitel (2017) style should be used.

VS Code can support adhering to a coding style. It can even restyle the code using the command *format document* (shift-alt-F Windows; opt-shift-F Mac), also in the context menu of the file.

Tuning the style can be done in the settings of VS Code in the setting C_Cpp: Clang_format_style with the value {BasedOnStyle: Google} whereby the tab size is 2 instead of 3.

5.4 File Comment Header

It is good practice to *always* put a standardised comment block at the top of each file, presenting basic administrative data and a short description of the contents / functionality of the file.

For the assignments it is important that each source file created by you (`*.cpp` and `*.hpp`) contains a comment header with the group number and the names of the group members.

Use the file comment header of Listing 5.1 in all your files.

Automatically generate a file header

Using a *Code Snippet* in VS Code to add the standard file header can be automated.

```
//==============================================================================
// Authors :
// Group   :
// License : LGPL open source license
//
// Brief :
//
//==============================================================================
```

Listing 5.1: Standard File Comment Header for all your source and header files. "Brief :" is the short description of the file contents (class).

Add the text of Listing 5.2 to the *cpp.json* snippets file, and update the two comment lines: the "====....====" strings must be 2 "/" and 98 "=" characters. Access this file via Settings > Configure Snippets and select the file. Via the shorthand *stdhdr* you can add the standard file header (snippet code) to your file at the cursor position while editing a c++ file.

```
"Standard Comment Header": {
  "prefix": "stdhdr",
  "body": [
    "//=====================.........................================================",
    "// Authors : ",
    "// Group   : ",
    "// License : LGPL open source license",
    "//",
    "// Brief : ",
    "//=====================.........................================================"
  ],
  "description": "Standard Comment Header for cpp and hpp files"
}
```

Listing 5.2: JSON code to automate insertion of the standard file header using VS-Code. Each comment line must consist of two "/" and 98 "=" characters between the " ". Here not all these "=" are shown to stay within the text width of this pdf.

Remarks and Tips

- prefix is the text string to type to get this code snippet inserted. It is shown by the suggestions generated by the Intellisense feature of VS Code.
- Adding your names and group number already in the definition of the code snippet, might be handy.

5.5 Build / Compile C++ projects

Building is the complete process of generating an executable from source files. For most C++ projects, building is actually compiling and linking, and can be started from the command line by running the compiler.

We present here *compiling* and *linking* from the command line, although VS Code supports this and hides it behind its graphical user interface.

Note that README files must contain a compile command that works on the command line, to allow those who grade your work can build the code on their own computer.

The compiling/building actions as presented here treats both forms of structuring files:

- *Source* files in folder src/ and *header* files in folder include/ as shown in Section 5.2.

- *All* files in *one* folder, so for small projects. This is done for the test programs in this chapter and the test program in Section 3.3.

Preparation

- Construct a file structure as explained in Section 5.2 and shown in Figure 5.1, so at least a src/ folder for the *.cpp files and an include/ folder for *.hpp files.
- Place the source and header files accordingly.
- For smaller projects, putting all files in one folder will work.

Standard Build/Compile

Go to the project directory, in which either the src/ and include/ folder are, or in which all *.cpp and *.hpp files are.

By default, the compiler does both compile and link actions in one command. The generic build/compile command is, with <program name> the name of your program executable:
g++ src/*.cpp -iquote include -o <program name>

Or in case all files are in the same directory:
g++ *.cpp -o <program name>

Remarks

- In Windows the extension .exe is automagically generated.
- Mac and Linux executables do not need a specific extension.
- As shown, the compiler accepts wildcards. In case you want to build a specific .cpp file, simply put that file name instead of *.cpp
- -iquote include indicates to *only* look for include files included with quotes (" "), so include files of the project itself, in that folder, in our case include. Works sometimes better than the regular include flag: -I include.
- More on include directories and order of looking up by the compiler is on Options for directory search webpage.
- If no -o flag is used, the default name of the resulting program is chosen: a.out

Other tasks with the compiler and specific settings

It is possible to let the compiler do either the compile step or the link step only, and specify other specifics.

- Compiling *only* is using the -c flag producing object files (*.o files), one for each C++ file:
  g++ -c *.cpp
- Linking *only* is by letting the compiler work with object files (*.o files) only, so:
  g++ *.o -o <program name>
- Specifying the C++ dialect, i.e. the language version, is by using the std=c++<nn> flag. Deitel and Deitel (2017) uses C++14 and C++17, but C++20 can also be used. Each newer C++ dialect has more features than the older ones, so a too new version might give compatibility problems. A command looks like:
  g++ -std=c++20 *.cpp -o <program name>
- Use -std=c++17 or -std=c++20 to avoid possible compatibility problems for the teachers and TAs while checking your code.
- Settings for running *multithreaded* code is only needed for Linux (Ubuntu), by using the -pthread -lpthread flags. However, this is not always needed.

To show the C++ version that is used by the compiler, you can query the long variable __cplusplus in a C++ program, for instance using this example.

5.6 Running and Debugging the generated program

Running the generated program is like running any other program from the command line:

Windows .\<program name>
Linux / Mac ./<program name>

Note the ./ or .\ before the program name. This is to indicate that the program is located in the current folder, and assuming that this folder is *not* in the list of folders the operating system looks for programs.

Debugging from the command line is doable, however rather unhandy. For debugging, using VS Code is far more convenient, see Section 3.6.2.

5.7 Operating-System specific system calls

Sometimes you might want to use a system call, a sleep function for example. When you use these functions, you have to be careful that the method of calling the function might be different on a different operating system. The sleep function on Windows requires you to include the <windows.h> header. However, the sleep function on Linux requires you to include the <unistd.h> header. This means that you need to be careful when using these kinds of functions.

In this case, a simple solution to make sure that the sleep function works on both systems can look as shown in Listing 5.3.

```
#ifdef _WIN32
    #include <Windows.h>
#else
    #include <unistd.h>
#endif
```

Listing 5.3: Example sleep function

5.8 Testing a multithreaded program

Here we present a test program that uses multithreading. It also serves as a typical example of a multithreaded program.

The example code is in Listing 5.4 giving the output: "The answer is 42".

Formatting is lost when copying the listing, but VS Code has an auto formatting function, which can be found using the command palette, reachable via Ctrl + Shift + P or Cmd + Shift + P for Mac and then searching Format Document. Do note that the key-combination to activate this function is also shown (if set).

5.9 Prepare for Hand-in

When handing in your assignments, your code should work on the PC of the grader, usually a TA. We have set some rules to make this grading easier. These rules align with common practice in software development, so supporting *good programming practice*.

5.9.1 Preparing for handing in a C++ project

You must submit *one* .zip file containing everything you produced to build and run your code independently. This zip is next to the report on the assignment, which is a separate .pdf file. Your submitted zip with code should look like Figure 5.1b.

Rules, Remarks, and Tips

- Keep the directory structure as you made during development.

```
#include <iostream>
#include <thread>

volatile int globalVar = 1;

void threadFunction(int x) {
    globalVar = x;
}

int main() {
    // Start a new thread that calls "threadFunction"
    std::thread t(threadFunction, 42);

     // Waits for the thread to finish
    t.join();

    // print results
    std::cout << "The answer is "
    << globalVar
    << std::endl;

    return 0;
}
```

Listing 5.4: Test code to see if the compiler is installed correctly

- Code provided by us *must* be excluded and *must* be described as a requirement in the top-level README.txt
- Remove any executables (e.g. assignmentX.1.exe on Windows, assignmentX.1 on Linux and MacOS).
- Remove any object files and other files generated by the compiler (e.g. assignmentX.1.o)

5.9.2 README files

Listing 5.5 shows an example README file. This README file contains clear instructions on how to compile your code (so exactly the command you used to compile), to run the program, and to work with the program.

```
Assignment X.1
-----------------------------------------------
1. Build command:
g++ src/main.cpp src/MyCoolClass.cpp -I ./include -o assignmentX.1.exe
2. Run command:
assignmentX.1.exe
3. Set focus on the opened window and control the character with W, A, S, D keys.
```

Listing 5.5: Example README file.

Notes

- The compile commands that you used contain parts specific for your OS, e.g. g++ and the .exe extension for the executable. This is not a problem as the TAs will adjust them to match their own host OS.
- For the README-file text, the obvious first steps like unzipping, placing the files in the standard folder structure, do *not* need to be written. This is also omitted in the example README file.

5.10 How graders/TAs grade your C++ projects

This section describes how the graders/TAs will grade your submitted work on C++ projects.

1. The grader downloads your submitted zip file from Canvas and unzips it.
2. The code will be copied into their own C++ project workspace, preserving the directory structure as you have provided in the zip file. *None* of the libraries we supply will be copied, as these are already present (the original ones).
3. The grader runs the compile command as specified in the top-level readme file, if needed updated to deal with differences in operating systems of your laptop and theirs, as stated in Section 5.9.2.
   - Any issues during this compile process can cause reduction of your grade.
   - Graders do *not* debug your work.
   - The top-level README should explain any steps how to run your work.
   - Graders have our provided libraries available.
4. Graders grade your work.
   - The design document (submitted separately as pdf) must contain your design decisions and all answers to questions in the assignment.
   - Your code must match the explanations in your design document. The grader checks this.
   - The grader uses the instructions given in your top-level README to run all code components. This README should also describe where something is implemented so that the grader can easily find it in your code. See also Section 5.9.2.
5. Based on answers and rubrics you are graded. Rubrics are available on Canvas.

6 Structuring, Documenting, Building, and Handing in ROS2 projects

6.1 Introduction

This chapter provides rules on how to properly structure, document, and hand in ROS2 projects. It extends Chapter 5 with respect to structuring, documenting, and code style. Doing so, your work is consistent with common approaches for ROS development, and is easier understandable by others, also the TAs.

The folder structure of ROS2 projects is explained in Section 6.2, whereby this structure is according the standard as used in the ROS2 community.

On documenting and structuring code of ROS2 nodes is in Section 6.3. Also, how to deal with the variation in coding styles used in the many available ROS2 packages / nodes.

The workflow on building ROS2 projects, also those with separate ROS2 nodes for the messages, is in Section 6.4. Running nodes and using Launch files is in Section 6.6.

The rules to hand in ROS2 projects are in Section 6.7, including rules about README files that are an essential part of documenting ROS2 packages / nodes. Examples of README files are in Section 6.9.

How TAs grade your work is in Section 6.8.

6.2 Structuring ROS2 projects

6.2.1 ROS Workspace

The *standard* structure of files (for instance, as a result of doing ROS2 tutorials) looks like Figure 6.1.

```
ros2_ws/
├── build/
│   └── (autogenerated colcon build files)
├── install/
│   └── (autogenerated colcon install files)
├── log/
│   └── (autogenerated colcon log files)
└── src/
    └── <packages>
```

Figure 6.1: Standard structure of a ROS2 project.

The ros2_ws folder is called the *base* of your ROS workspace. It is the real starting point of all the work in your ROS workspace:

- To start the building / compilation process via the command colcon build
- To configure the environment using source install/setup.bash
- To put all work (source files, configuration files) in ros2_ws/src/

We do *not* discuss the content of the *autogenerated* build/, install/, and log/ folders, obviously.

6.2.2 Packages

During development, code is organised in packages. These contain one or more ROS nodes, or message definitions. These aim to separate different components, such as the cam2image

and relbot_simulator packages described in Chapter 7 and Section 15.2 respectively. Messages are also kept separate, such that they can be compiled separately from the package that needs them. Putting this all together looks like Figure 6.2.

```
ros2_ws/
├── build/
├── install/
├── log/
└── src/
    ├── cam2image
    ├── relbot_simulator
    └── message_definition
```

Figure 6.2: Structure of ROS project with several packages.

Within these folders, code files are separated by function such as source files in the src/ and header files in the include/ folders, as usual for C++ code. The same is done for supporting ROS files such as config/ and launch/ files. See Figure 6.3 for an example. This structure is also used for messages, to keep consistency between all source files (Figure 6.4). Both have package.xml, CMakeLists.txt, and a README.txt. The README.txt describes the contents of the package on its own. See Section 6.7.2 for requirements when handing in.

```
<package name>/
├── config/
│   └── config.yaml
├── include/
│   └── filename.hpp
├── launch/
│   └── node.launch.py
├── src/
│   └── filename.cpp
├── CMakeLists.txt
├── package.xml
└── README.txt
```

Figure 6.3: ROS Node package.

```
<message package name>/
├── msg/
│   └── MyMsg.msg
├── CMakeLists.txt
├── package.xml
└── README.txt
```

Figure 6.4: ROS Message package.

6.3 Commenting and Coding Style of ROS2 nodes and support for it by VS Code

Existing ROS2 nodes have their "file header" documentation in the package.xml file of the package. It has places for *description*, *maintainer e-mail*, and *licence*. The mail address is of the *maintainer* and not the owner, as the maintainer would react to questions and repair / update requests. It is good practice to fill these in. For the other documentation parts (per declaration, method, inside method), *Doxygen* tags are (often) used. Furthermore, developers often rely on well-thought-out descriptive names of variables and functions / methods, and do *not* add comments to those declarations.

For your ROS2 nodes, produce documentation as follows:

- Fill the *description* and *maintainer e-mail* parts in the package.xml file of the package (replacing the "file header" information compared to C++ projects). Authors and maintainers are considered now the same persons.

- Add comments per declaration, per method and inside methods in case that piece of code is quite long or has some difficult coding constructs.
- Put design decisions and ROS2 node graphs (preferably using RQt) in your report.

One of the most notable differences between coding styles is the placing of the opening brace { of a function or class: on the same line as the signature as is used by Deitel and Deitel (2017) or separately on the line directly below the signature (used in quite some packages provided by the ROS2 community).

For ROS2 nodes the Deitel and Deitel (2017) style should be used, although different existing ROS2 nodes often have (slightly) different styles (and not always the same style). Adhering to such a ROS2-like style for your coursework is acceptable. Being consistent in using one style while developing your code is more important than the specific style.

VS Code can support adhering to a coding style. It can even restyle the code using the format document command (shift-alt-F Windows; opt-shift-F Mac), also in the context menu of the file.

Tuning the style can be done in the settings of VS Code in the setting C_Cpp: Clang_format_style

- {BasedOnStyle: Google} for Deitel and Deitel (2017)-style formatting, using tab size = 2.
- {BasedOnStyle: Mozilla} for ROS2-style formatting.

6.4 Building ROS2 projects

The compilation/building workflow as presented here assumes that the files are structured in a workspace as presented in Section 6.2. Besides adhering to ROS2 standards, and thus easier to understand by others, including graders and TAs, also the ROS2 tools to be used here assume this way of structuring.

So, it is really advantageous to adhere to the ROS2 standards with respect to structuring files in your work space.

Preparation

- Put each package in a separate folder in the src/ folder of your workspace, according to Figure 6.2. This also holds for the packages provided by the Robotics Education Lab.
- Inside the packages and message packages, structure the files according to the ROS2 community standard as shown in Figures 6.3 and 6.4.
- **Note**: Put all include files in the include folder, so *not* in the <package name> subfolder of it, as is produced by the ros2 pkg create command. See the provided packages relbot_simulator and relbot_adapter.

Building/Compiling

In case your project has message packages, these need to be compiled first.

- Be in the *base* of the workspace, ros_ws as used in this manual.
- If there are any message packages, build them first and set the (updated) environment variables (mind the *two* dashes before the packages-select argument):
  - colcon build --packages-select <message package(s)>
  - source install/setup.bash
- Build *all* code using the standard colcon command and set again the updated environment variables:
  - colcon build
  - source install/setup.bash

After this building process has finished successfully, testing of the packages can start.

Tips and Remarks

- To exclude a folder (package) from being compiled, add an (empty) file called COLCON_IGNORE in the src/ folder of that project. Use the touch command for this in that src/ folder: touch COLCON_IGNORE
- In case errors are persistent even though the code was fixed, the cause might be auto-generated files being *not* updated. A solution is often to remove those:
  rm -r build/ install/ log/
- *Always* run the set (updated) variables command after a colcon build.
- See Section 12.9 for more tips on colcon itself.

6.5 Running nodes

Nodes can simply be started using the command line using the ros2 run command, here with the optional ROS2-arguments flag (mind the *two* dashes before the ros-args argument):

```
ros2 run <package> <node> [--ros-args -p <param name>:=<value]
```

In case quite some nodes need to be started, it is beneficial to construct and use a *launch* file in which the nodes including configuration information are listed. Seen the next section.

6.6 Launch files

A versatile way to start a ROS2 program consisting of several nodes with configuration info via arguments is to use *launch* files.

We prefer to write launch files in python, as that gives more flexibility than writing launch files in XML or YAML. It is a bit more verbose, though.

6.6.1 Structure of launch files

The structure we use is different from those in the ROS2 tutorials. In our case, the nodes are described first, and in the return command (the *LaunchDescription*) all the nodes to be used are specified. This to have a more structured overview than used in the ROS2 tutorials[1]. An example is in Listing 6.1.

Often remapping of output messages of a node to input messages of the connected node is needed. Before specifying this in the launch file, it is beneficial to make a simple block diagram to show these remapping. Put in such a figure the parameters *above* the nodes, and the remapping *below* the nodes, as shown in Figure 6.5.

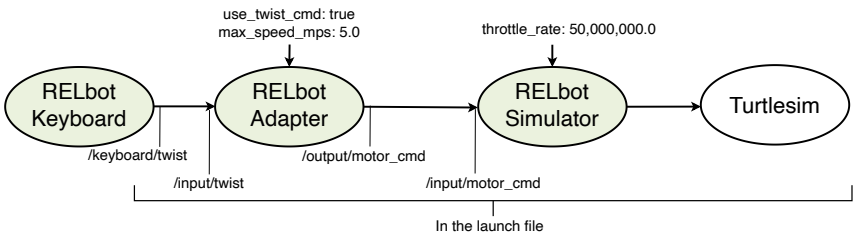

Figure 6.5: Node structure while setting up the launch file. Annotations: parameters above the nodes; remapping of messages below the nodes. In this case, it is for the RELbot simulation using keyboard input. The keyboard node *cannot* be started from within a launch file, hence the annotation at the bottom; see Section 15.5.

6.6.2 Running the launch file

In a terminal run the following command assuming all nodes have been built correctly:

[1] See the ROS2 tutorial Launching nodes and in the 'chapter' on Launching ROS2 nodes: Launch.

```python
from launch import LaunchDescription
from launch.substitutions import LaunchConfiguration
from launch.actions import DeclareLaunchArgument
from launch_ros.actions import Node

def generate_launch_description():
    adapter_node = Node(
        package='relbot_adapter',
        executable='relbot_adapter',
        name='relbot_adapter',
        parameters = [
            {"use_twist_cmd" : True},
            {"max_speed_mps" : 5.0},
        ],
        remappings=[
            ('/output/motor_cmd' , '/input/motor_cmd'),
            ('/input/twist', '/keyboard/twist')
        ]
    )

    relbot_simulator = Node(
        package="relbot_simulator",
        executable="relbot_simulator",
        name="relbot_sim",
        parameters=[
            {"throttle_rate": 50000000.0},
            ],
    )

    relbot_turtle = Node(
        package="turtlesim",
        executable="turtlesim_node",
        name="robotturtle"
    )

    return LaunchDescription([
        adapter_node,
        relbot_simulator,
        relbot_turtle
    ])
```

Listing 6.1: Launch file of a RELbot simulator project, using our structured approach

```
ros2 launch <launch-package name> <launch file name>
```

Tips and Remarks

- It really pays back to make on beforehand a diagram of your design of the launch structure as in Figure 6.5.
- Check whether the running nodes are consistent with your planned node structure by producing the RQt graph of the running nodes, and compare that with your design.
- Putting the node structure diagram and the RQt graph of the node structure in your documentation helps to understand the launch file by reviewers and graders, and it shows that the designed structure really works.
- Put the launch file in the directory launch in the package directory, as shown in Figure 6.3.

- *Remark*: In large systems, launch files that use nodes from several packages can be placed in a separate package consisting of a directory launch containing those launch files.

6.7 Preparing a hand-in for ROS2 projects

When handing in your assignments, your code should work on the laptop of the grader, usually a TA. We have set some rules to make this grading easier. These rules align with common practice in ROS2 node/software development, so supporting *good ROS2 programming practice*.

6.7.1 Hand in contents

You must submit *one* .zip file containing everything to build and run your code independently. This next to the report on the assignment, which is a separate .pdf file. The automatically generated build/, install /, and log/ folders should *NOT* be included. Your submission should look like Figure 6.6.

Code provided by us *must* be excluded and *must* be described as a requirement in the top-level README.txt

```
assignment.zip
├── my_custom_package/
│   └── README.txt (package level)
├── my_other_custom_package/
│   └── README.txt (package level)
├── my_custom_messages/
│   └── README.txt (package level)
└── README.txt (top-level)
```

Figure 6.6: Project contents ready for submission as zip file.

6.7.2 README files

For an assignment containing ROS work, you need two types of README files:

- Top-level README: to explain how to run all the packages in the project, ordered per subassignment. See an example in Listing 6.2 in Section 6.9.
- Package README: to describe the package and how to use it. Every package is self-contained and therefore should contain a README file. See an example in Listing 6.3 in Section 6.9.

For the README-file text, the obvious first steps like unzipping, placing it in the standard folder structure (so like in Figure 6.2), and compiling / building using the colcon build command do *not* need to be written, as is also omitted in the two example README files. So, steps 1 – 3 in Section 6.8 do *not* to be mentioned.

6.7.3 Submission instructions

Before each submission:

- Make sure *your* packages work in a clean environment. This can be done by removing the build/, install /, and log/ folders, rebuild it, and remove those folders again. Or *copy your* packages to be submitted in fresh workspace that includes the provided packages and test that one.
- Make sure that each package includes a *package-level* README.txt file.
- Make sure that the *top-level* README.txt file describes how to run each sub-assignment. Assume ROS2 Humble is installed.

- Test the generated zip file and various README.txt files on a different installation, such as your group-mates laptop.

Assuming everything above passes, select the top-level README, all *your* packages related to the assignment in the ros2_ws/src/ folder and zip them.

Note: Do *NOT* include *provided* packages in the zip file! This to minimise the number of files in the zip but more importantly, be sure that you did not accidentally have modified the provided packages. Furthermore, it is easier for the graders/TAs as they do not get the provided packages again for every group.

6.8 How graders/TAs grade your ROS2 projects

This section describes how the graders/TAs will grade your submitted work on ROS2 projects.

1. The grader downloads your submitted zip file from Canvas and unzips it.
2. The packages will be copied into their own ROS workspace, similar to Figure 6.2. *None* of the packages we supply will be copied, as these are already present (the original ones).
3. The grader runs colcon build and source install/setup.bash from the base of their ROS workspace, or any specific build commands as written in the top-level README file.
   - Any issues during this build and configure process can cause reduction of your grade.
   - Graders do *not* debug your work.
   - The top-level README should explain any steps how to run your work.
   - Graders have ROS2 Humble desktop and our provided packages installed.
4. Graders grade your work.
   - The design document (submitted separately as pdf) must contain your design decisions and all answers to questions in the assignment.
   - Your code must match the explanations in your design document. The grader checks this.
   - The grader uses the instructions given in your top-level README to run all code components. Any command will be run from the base of the workspace. This README should also describe where something is implemented so that the grader can easily find it in your code. See also Listing 6.2 in Section 6.9.
5. Using rubrics you work is graded. Rubrics are available on Canvas.

6.9 Examples of README files

- A *top-level* README file example of an imaginary assignment in plain *text* format is in Listing 6.2. It must be placed in the *top-level* src folder of the ROS2 workspace, so the file is src/README.txt.
- A *package* README file example is in Listing 6.3, in *markdown* format. It is the README of the RELbot_Simulator node. The file is in the folder src/relbot_simulator/, so the file is src/relbot_simulator/README.md. See Section 15.2 for the explanation of this node.

Remarks and Tips

- It is *not* necessary to write README files in markdown format, although it is common use in ROS development. Plain text (in .txt files) is accepted for assignments.
- VS Code has a previewer for markdown format, and quite some (open source) markdown editors exist.

```
Assignment 25.1
-----------------------------------------------
1. Run
    ros2 run my_custom_package node1
    Location in code: light_detection () function in image_analysis.cpp

Assignment 25.2
-----------------------------------------------
1. Open two new terminals, in the first run
    ros2 run my_custom_package node2
    Location in code: colour_detection() function in image_analysis.cpp

    In the second run
    ros2 run my_other_custom_package node1
    Location in code: steer_calculation () function in steer ()

Assignment 25.3
-----------------------------------------------
1. Run
    ros2 launch my_custom_package start_all_nodes.launch.py

    and send a velocity
    ros2 topic pub /my_topic example_msgs/msg/Float64 "{data: 2.0}"

    Assignment connects node 1, 2 and node from other-package together.
```

Listing 6.2: Example top-level README file (in text format).

```
Package relbot_simulator
-----------------------------------------------
### Description
This package implements a simulated relbot. Model generated by 20-sim

### Inputs
'/image'
      Type: sensor_msgs/msg/Image

'/input/motor_cmd'
      Type: relbot_msgs/msg/RelbotMotors
      Combined float message. Exists of 'left_wheel_vel' and 'right_wheel_vel'

### Outputs
'/output/camera_position'
     Type: geometry_msgs/msg/PointStamped
     X,Y position of camera centre on input image

'/output/moving_camera'
     Type: sensor_msgs/msg/Image
     sub-image of input. Depends on X/ thetaZ of relbot position

### Run
In a terminal run the following commands:
'ros2 run relbot_simulator relbot_simulator'

### Parameters
double 'image_stream_FPS' : Sets the output rate of image stream. Default = 30 FPS, which most webcams have
double ' throttle_rate ': sets throttle rate of ros_log messages. Only publishes first message after ' throttle_rate ' amount of ms
      has passed.
double 'visualisation_frequency': sets the output frequency towards turtlesim. Default 62.5

### Core components
* 'dynamics_timer_callback()': Calls and runs the model every simulation timestep
* 'webcam_topic_callback()': Ingests and passes on webcam image stream
* 'CreateCVSubimage()': Creates sub-image which is then published on /output/moving_camera
```

Listing 6.3: README file of node RELbot_Simulator (in markdown format).

7 Webcam stream to the VM

Using a VM means that direct access to your webcam is a bit more complicated. Our cam2image_vm2ros package makes this easily possible. It is an adapted version of the standard cam2image available in the ROS community.

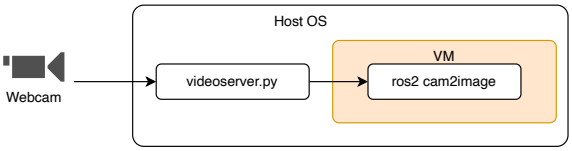

Figure 7.1: High-level overview of how the webcam stream is passed to the VM for further use there.

Prerequisites

- Have a recent version of Python 3 installed.
  - If you use python for other projects, a venv (virtual environment) is an option. See documentation for instructions to set up the venv functionality.
- Have your host-side IPv4 address of your VM connection at hand (router address of the internal local network between host and VM).
  - **Mac:** Find this by running ifconfig | grep 192 and find number after inet.
  - **Windows:** You can find this by running ipconfig and search for the entry with WSL in the line. You can find your IP adress under IPv4 address.
- **For Mac:** Increase the UDP data-frame size to the maximum by running sudo sysctl -w net.inet.udp.maxdgram=65535 . Running this has no adverse effects on your machine. To reset, run the same command with 9216 as maxdgram.

Host machine

1. Place cam2image_host2vm anywhere on the filesystem of your host computer.
2. Open a terminal in this folder, make sure all required python packages are installed by running pip install -r requirements.txt
3. Run the videoserver.py by calling python videoserver.py. Some systems might use python3 as command. Command autocomplete with tab should help.

VM

1. Place cam2image_vm2ros in the src folder of your ROS workspace.
2. In a terminal, run colcon build in the base of your ROS workspace.
   - See also Figure 6.2 for layout.
3. Set up the environment by running source install /setup.bash
4. Edit the config/cam2image.yaml file in the cam2image_vm2ros package, change the socket_ip line to your host IP.
5. Run the package by running ros2 run cam2image_vm2ros cam2image --ros-args --params-file src/cam2image_vm2ros/config/cam2image.yaml

Note: the videoserver.py script should be running before starting the node on the VM!

Debugging parameters

- show_camera will show the camera image over an ssh -X connection. WSL automatically does this, but Multipass and the Raspberry Pi need the -X argument!
- remote_timeout sets the amount of seconds the node waits before timing out. Default = 7

8 Accessing the Raspberry Pi of the RELbot

The Raspberry Pi of the RELbot is prepared to be used by several student groups, and each group has its own login and files. This is just like any other system running Ubuntu.

8.1 User name/account

Each group has their own account, with a private password. Login name and password are provided by the teacher or TAs.

You can change the password if you want but be aware that it could be reset as well (see Section 8.3). Each group has his own encrypted home directory on each Raspberry Pi.

Note: The home directories are local on each Raspberry Pi, so when you log in on another Raspberry Pi you will *not* find your files there produced during a previous session on another Raspberry Pi.

8.2 Connecting to the Raspberry Pi (SSH)

Access to a RELbot is via the Virtual Private Lan (VPN) to which the Raspberry Pi of the RELbot is connected. In this chapter, it is called *RELbot-VPN*. You use your own laptop as the development PC. In order to connect to the Raspberry Pi, do the following:

1. Make sure that the Raspberry Pi is turned on.
2. Choose one of the following options:
   - With your own laptop, connect via WiFi to the RELbot-VPN[1], using the given password. When on the RELbot-VPN network, you do *not* have connection to the internet.
   - When in the RaM-Lab, connect your own laptop to the RaM-lab VLAN with a *green* Ethernet cable[2]. This way your WiFi connection is still free to simultaneously connect to a WiFi access point providing internet (e.g., Eduroam or EnschedeStadVanNu).
3. Access the Raspberry Pi via SSH as described below.

Obviously, you are only allowed to log in onto the Raspberry Pi on your own lab table. Each Raspberry Pi shows on its display its IP address, e.g., 10.0.28.109). Logging in onto a Raspberry Pi goes through the SSH protocol; you need an SSH-client program on your own laptop to do that.

Log in on the RELbot's Raspberry Pi via the ssh command:
ssh [-X] <username>@<IPv4 address RELbot>
see Section 4.8.3 for more information, and showing graphics being produced on the RELbot on the screen of your laptop using the -X argument.

Note: Multiple SSH connections to the Raspberry Pi at the same time are possible, e.g., one for running your program, another for running the load-generating program and a Visual Studio Code connection.

8.3 Avoid loosing files

It is important to copy your files to your host (laptop) when your time slot has ended.

In rare circumstances, the SD card of the Raspberry Pi can become corrupt. In such a case, the TAs can decide to wipe the SD card and start with a fresh install. Then all user files on the

[1] In the RaM-Lab in the Carré building, the VPN is called RaM-lab, with password *robotics*.
[2] The RaM-Lab provides two separated networks: the UT net (red cables) and the RaM-lab VPN (green cables).

Raspberry Pi are irreversibly lost, and *all* your files on the Raspberry Pi get lost. See the Tips in Section 4.6. FYI: the file synchronisation tool git is installed on the Raspberry Pis.

8.4 Remarks and Tips

- Use your own laptop for developing C++ programs and editing files remotely with Visual Studio Code; see Chapter 4, especially Section 4.5.
- For editing files (scripts, ROS2 parameter files, etc) directly on the Raspberry Pi, VS Code, remotely connected, can be used of course.
- Editing files directly on the Raspberry Pican also be done using a straightforward text editor running on the Raspberry Pi, using a ssh connection from your host machine. The often-used editors nano and vim are available on the Raspberry Pi.
- No GUI/Windows manager is installed on the Raspberry Pis.
- To copy your files you can use the command line tool scp or use a GUI sftp/scp program like winscp. Or git as synchronisation tool as mentioned above.

9 XRF2, the Xenomai 4 – ROS 2 framework

The Xenomai 4 – ROS 2 Framework, second generation (XRF2) provides a skeleton to set up data communication between a ROS 2 node and code running firm real-time on a Raspberry Pi core with the Xenomai 4 / EVL[1] operating-system extension. Background information and rationale for the design of XRF2 are in (Raoudi, 2024).

XRF2 provides three pieces of functionality (see Figure 9.1):

- Communication software to let ROS2 programs and Xenomai 4 programs communicate with each other, while adhering to both the ROS 2 structure and respecting the firm real-time timing demands of Xenomai 4 at the corresponding side of the communication.
- A skeleton for code to run in firm real-time on Xenomai 4 / EVL in a while loop, with the computation part to execute (preferably) generated by 20-sim, and a Finite State Machine (FSM) to coordinate which computation part must run in which *logical* state of the robot.
- Firm real-time code to monitor and log data in the FRT loop.

Note: The controller function must be called LoopController().

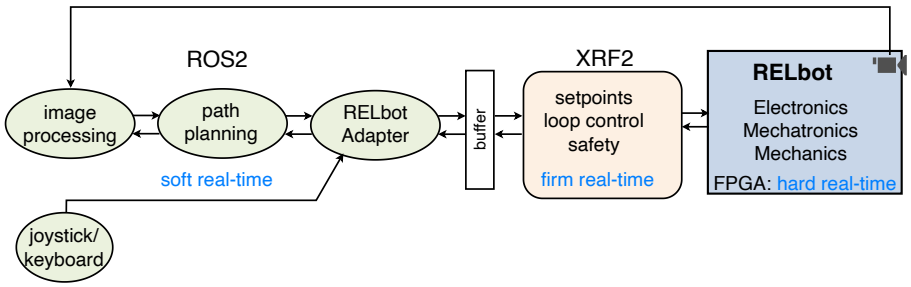

Figure 9.1: RELbot software components connected to the mechatronics. XRF2 provides the buffer with its interfaces and the firm real-time framework. Monitoring parts are not shown. This figure is a recall of **??**, bottom part.

The overview of all code running on the RELbot is shown in Figure 9.2.

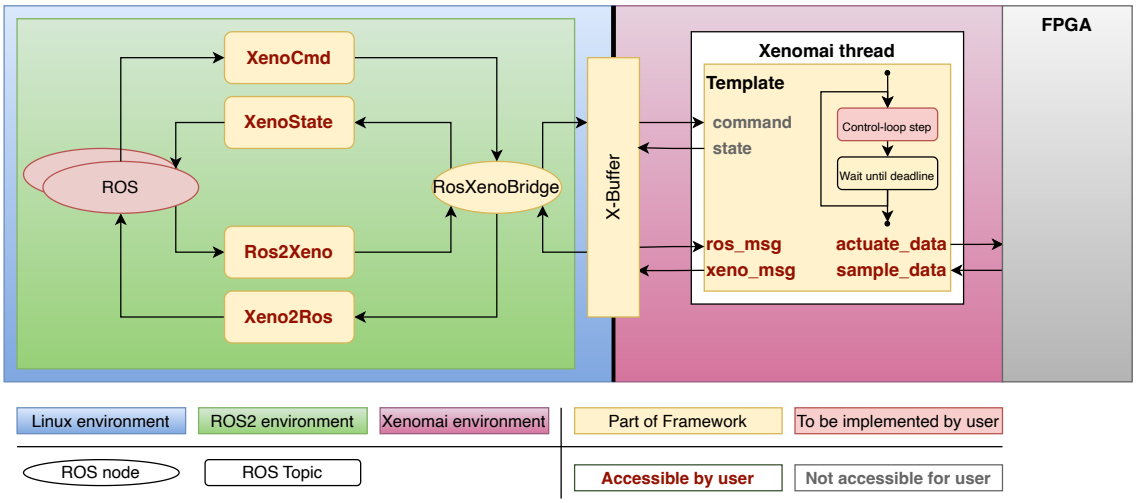

Figure 9.2: Typical use of XRF2 connecting a ROS 2 node and a Xenomai program. Code blocks annotated in red must be adapted / provided. Only messages and variables in red are accessible. Adapted from Raoudi (2024, Fig. 3.8).

[1] The names Xenomai 4 and EVL are used interchangeably, also in the documentation by the developers.

Testing your software for the RELbot using the simulators first (see **??**, 15.2, and 16.2) is explicitly supported. Doing so, gives you a flavour of working along approaches of *model-based software development*, which is currently *the* way of developing embedded / robotic software in high-tech industry.

This chapter is about using the the XRF2 framework itself. The firm real-time monitoring and logging provided by the XRF2 framework is presented in Chapter 10.

9.1 Structure of the Framework

The file structure of the framework is shown in Figure 9.3.

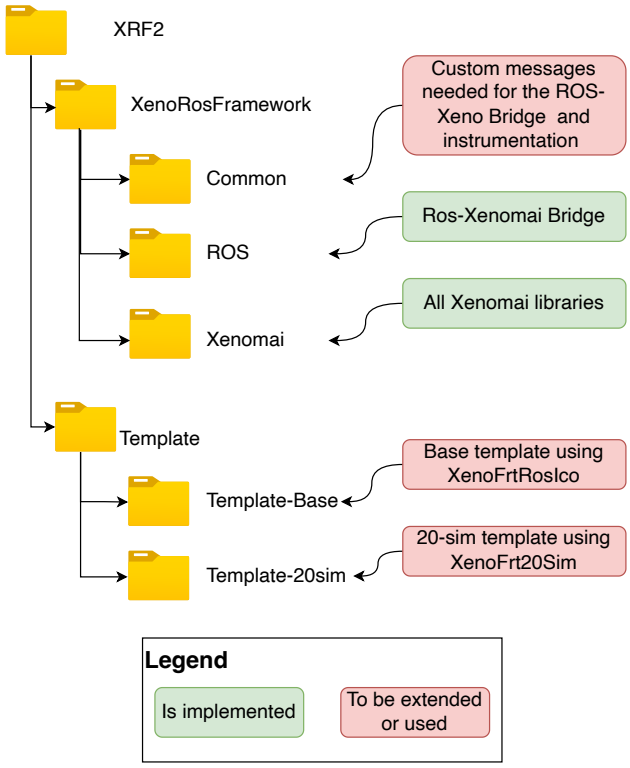

Figure 9.3: File structure of the framework. Files in red annotated folders must be updated. Adapted from Raoudi (2024, Fig. F.1).

9.2 Using the framework in your project

The framework must be downloaded on the RELbot before it can be used.

Use the following step-by-step instructions on how to use the framework and to extend / fill the template for the FRT loop, and to build and run the code. Some background information is given in Section 9.3.

Preparation

- Download XRF2, uncompress it and put the framework in the `src` folder of your ROS2 workspace (`ros2_ws`) on the RELbot.
- Copy one of the templates from the `XRF2/Template` folder in the `src` folder of your workspace. Keep the file structure of that template intact.
- Rename the copied template to the name of your project part that runs Firm Real-Time on Xenomai 4.
- Your folder tree should look like Figure 9.4.

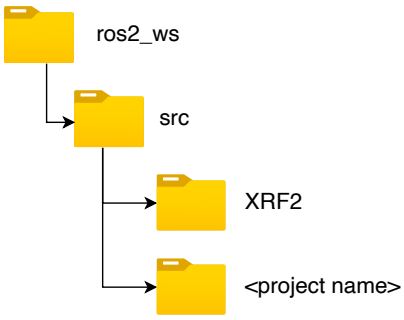

Figure 9.4: Top-level of your workspace after Preparation.

Messages

- Go to the folder XRF2/XenoRosFramework/Common/XRF2_msgs/msg .
- Change the messages and types in the two msg files to fit your project's needs. One message file per direction of the messages forming the data stream between the ROS2 side and the Xenomai 4 side. These names will be used in your code.
- Build this message package via
  colcon build --packages-select xrf2_msgs
- Set the (updated) environment variables using the usual command:
  source install /setup.bash being in the root or your workspace.

Incorporate the code generated by 20-sim

- The controller submodel from which 20-sim generates code must be named LoopController(), such that the name of the generated class is also LoopController. Indeed, hardcoded.
- Go to the folder ros2_ws/src/*<project name>*
- Fill the template with 20-sim generated code, in the folder controller as follows:
  - Remove all files in this directory, as these are skeleton files, and 20-sim produces new files.
  - Generate 20-sim code from your controller model and copy it to this folder.
  - **Note**: The class of the generated code must have the name LoopController.
  - **Note**: *All* of the code in this folder is generated, so do *not* edit these files.

Finite State Machine

First update the names of the project and files, then edit the header and cpp files of the FSM, shown in Figure 9.5.

- Go to the folder ros2_ws/src/*<project name>*
- Open the CMakeList.txt file and rename the original project name (tempate_20sim) to the name of your project in several places. A search-and-replace action would be beneficial here, using the built-in search-and-replace function in the Activity Bar of VS Code.
- In the folder include rename the header file Template.hpp to match with your project name.
- Edit this .hpp file as follows:
  - Rename the class and (de)constructor to match with your project and thus file name.
  - If the logger *is* used see Chapter 10 about how to use it.
  - if the logger is *not* used, remove the the struct GenericLogStruct.
  - Change the size of the arrays u and y to the correct size specified by your 20-sim-generated controller code.
- In the folder src rename the .cpp file to match with your project / header file.

- Edit this .cpp file as follows:
  - Remove all code lines *in* the functions implementing states that you are *not* going to use. Do *not* remove the return statements in these state functions.
  - Specify the actions / code of the states you need in its corresponding function. These are the states *Initialising*, *Run*, *Pausing*, and *Stopping*, as shown in Figure 9.5.
  - **Note**: The *Run* state function is prefilled with a call to the LoopController function that is generated by 20-sim.

Tweak the Computation in the *run* state function

The 20-sim model is already imported and is called controller. Through the controller.calculate function one computation step is performed.
The controller.finished function indicates when the simulation is finished.

Tip: Use monitor.printf() instead of evl_printf() for debug messaging to reduce the output flow of the printf statements to a lower frequency. It *is* an FRT-friendly function.

- Go to the folder ros_ws/src/*<project name>*
- Edit main.cpp and replace the template class with your own class.
- On line 13 the object of the class is made. The parameters of the class are:
  1. Communication frequency to ROS (Hz)
  2. Monitor frequency (Hz) of the monitor.printf()

Build and Run

- If not already done add all your ROS2 projects to the ros2_ws/src/ folder.
- Go to the base of your workspace (folder ros2_ws).
- Build all code present in this workspace (in /src) via colcon build
  To exclude a project from being compiled add an (empty) file called COLCON_IGNORE to the source folder of that project.
- Set the (updated) environment variables using the usual command from the root of your workspace:
  source install/setup.bash
- ***Always*** start the Xenomai project first, via:
  sudo ./build/*<project name>*/*<project name>*
- Start the ROS-Xenomai bridge in a different terminal being in the root of your workspace:
  ros2 run ros_xeno_bridge RosXenoBridge
  Do not forget to set the environment variables first via source install/setup.bash
  - **Note**: The Xenomai 4 project has to be started first in order to let it work.
- Start other ROS2 nodes in separate terminals, as usual:
  ros2 run <package name> <node name>
- Initialise the FSM and thus to start the *computation* at the Xenomai side by publishing the *initialise* command only *once* to the XenoCmd topic. This can be done in several ways.
  - Simply send the message from the commandline to the the topic:
    ros2 topic pub –once /XenoCmd std_msgs/msg/Int32 "{data: 1}"
    You cannot use the enumerated type as presented in Table 9.1, but use its integer value instead.
  - Send the message in the initialising part of a ROS node that deals with the start up. Waiting for the FSM to respond by listening to the XenoState topic for the value "3", *initialised*, before continuing is relevant here. See Table 9.2 for a complete list of FSM-state information.

Tips and Remarks

- In case initialising the FPGA does not succeed at the first attempt, restart both programs. You could also run one of the test programs, for instance, relbot-motortest, as first attempt to initialise the FPGA. This is a known inconvenience, unfortunately.
- Create a ROS2 launch file to make it easy to start all needed ROS2 nodes and topics. This reduces the amount of terminals needed. Commands to publish data to certain topics can also be added to launch files.

9.3 Background Information on XRF2

9.3.1 Finite State Machine

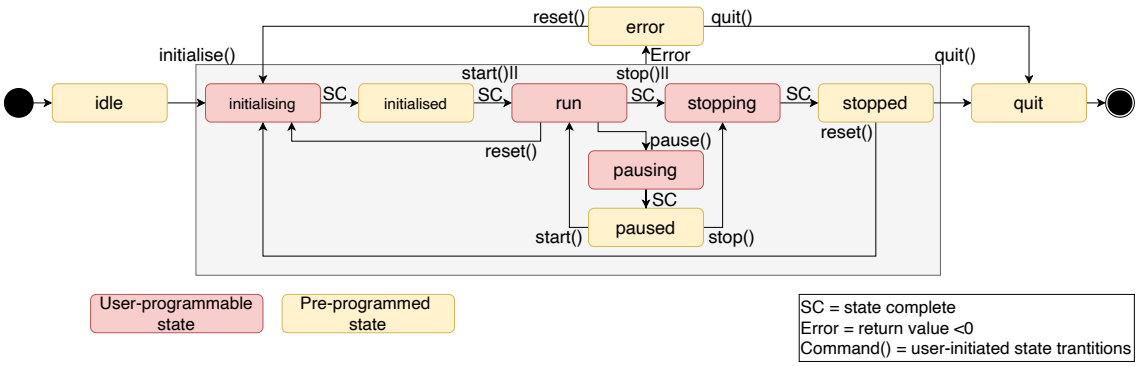

Figure 9.5: Diagram of the state machine as implemented in XRF2. From Raoudi (2024, Fig. 3.10).

Each function in the class, except the constructor and deconstructor represents a state. The state diagram is shown in Figure 9.5. The state machine can be controlled by the ROS2 topic XenoCmd. The resulting state value is published in the XenoState topic. To change the state of the FSM, publishing the integer value representing the new state to the XenoCmd topic. The commands are shown in Table 9.1 and the resulting states are shown in Table 9.2.

Table 9.1: Commands that can be given through the ROS2 topic XenoCmd to the FSM on the Xenomai4 side.

Command	Value	Description
initialise	1	From idle to initialising
start	2	From initialised or paused state to run
stop	3	From run or paused state to stopping
reset	4	From run, stop, error or pause to initialising
pause	5	From run state to pausing
quit	6	From error or stopped sate to quit. After this, the Xenomai side shuts down

Table 9.2: States reported back by the FSM to the ROS2 side on the ROS2 topic XenoState, when the FSM has arrived into the reported state.

State	Value
Idle	1
Initialising	2
Initialised	3
Run	4
Stopping	5
Stopped	6
Pausing	7
Paused	8
Error	9
Quit	10

9.3.2 FPGA data

The variable sample_data provides all the sampling data from the FPGA and has the following structure:

```
struct IcoRead {
    int channel1;    // Encoder value from channel 1
    int channel2;    // Encoder value from channel 2
    int channel3;    // Encoder value from channel 3
    int channel4;    // Encoder value from channel 4
    bool channel1_1; // Digital input 1 from channel 1
    bool channel1_2; // Digital input 2 from channel 1
    bool channel2_1; // Digital input 1 from channel 2
    bool channel2_2; // Digital input 2 from channel 2
    bool channel3_1; // Digital input 1 from channel 3
    bool channel3_2; // Digital input 2 from channel 3
    bool channel4_1; // Digital input 1 from channel 4
    bool channel4_2; // Digital input 2 from channel 4
};
```

The variable for controlling the FPGA output is called actuate_data and its structure is:

```
struct IcoWrite {
    int16_t pwm1;   // PWM value for channel 1
    bool val1;      // Digital output 1 of channel 1
    int16_t pwm2;   // PWM value for channel 2
    bool val2;      // Digital output 1 of channel 2
    int16_t pwm3;   // PWM value for channel 3
    bool val3;      // Digital output 1 of channel 3
    int16_t pwm4;   // PWM value for channel 4
    bool val4;      // Digital output 1 of channel 4
};
```

9.3.3 Controller data

As specified in the package XRF2_msgs, the data sent from ROS2 to Xenomai is called ros_msg and has the structure of the Ros2Xeno message. The data to be sent from Xenomai to ROS2 is set in the variable called xeno_msg and has the structure of the Xeno2Ros message.

The message files are in XRF2/XenoRosFramework/Common/XRF2_msgs/msg

9.3.4 UML diagram of the framework

The class diagram of the framework is shown in Figure 9.6.

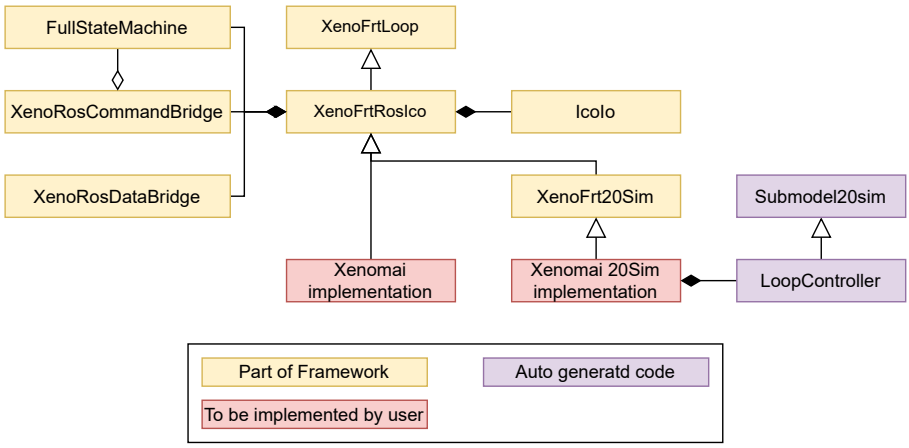

Figure 9.6: UML Class diagram of the framework (Raoudi, 2024, Fig. 3.11). One of the two parts in red must be extended.
UML symbols, so the 'arrows' at the lines between the classes (boxes): open triangle denotes inheritance ("is a" relation), more general at arrow-head side; the filled diamond denotes composition ("has a" relation), the strong ownership, owner at arrow-head side.

The variables representing the connections between the Xenomai loop and the ROS2 nodes respectively the motors (via the FPGA) are declared in XenorFrtRosIco.hpp, part of XRF2. Basic use of these variables is in the Template-Base.

10 Using the XenoFrtLogger

XenoFrtLogger provides methods to firm real-time log data on Xenomai when the control loop is running. This is done through a proxy that communicates the data from the Xenomai to the Linux kernel to log the data on a file. This class works on both the Xenomai and the Linux kernel. It is developed by Raoudi (2024).

The logger works by having one variable to log data from. This variable has to be defined by the user. Each element in the variable has to be registered. Start and stop commands can be given to determine when the logger has to log the data.

Below, step-by-step instructions are given on how to implement and use the XenoFrtLogger. Under each step, a code example is shown.

1. Create a variable that is going to be logged. This variable can consist of multiple elements like an array or struct. When using structs make sure to use "#pragma pack (1)" before the struct and "#pragma pack (0)" after the struct. This will remove the space within the memory between different elements. In the definition of the struct, default values can be given.

```
// Definition of the struct ThisIsAStruct
// A normal struct name is PosistionData
#pragma pack (1)
struct LoggedVariable {
    int this_is_a_int;
    double this_is_a_double;
    float this_is_a_float;
    char this_is_a_char;
    bool this_is_a_bool;
};
#pragma pack(0)

// Variable that is going to be logged
struct LoggedVariable data_to_be_logged;
```

2. Go into the main function and create a XenoFileHandler instance which represents the log file: file = XenoFileHandlerfile(1,"/home/pi/Logger_Test","bin")
3. Create the XenoFrtLogger instance: logger = XenoFrtLogger(&file, &data_to_be_logged)
4. Register all elements in the variable to the logger, using the addVariable function. Table 10.1 shows all the variable types that can be registered. The registration of the elements has to happen in the same order as the allocation in the variable.

```
logger.addVariable("this_is_a_int", id_int);
logger.addVariable("this_is_a_double", id_double);
logger.addVariable("this_is_a_float", id_float);
logger.addVariable("this_is_a_char", id_char);
logger.addVariable("this_is_a_bool", id_bool);
```

Table 10.1: Different constants of the enum VariableTypes.

Name	Size	Type
id_bool	1	bool
id_char	1	char
id_float	4	float
id_int	4	int
id_double	8	double
id_long_int	8	int64, timespec.tv_sec, timespec.tv_nsec

5. After having added all elements, initialise the logger once. If done twice, the program will crash. A check can be done to avoid this.

```
if (!logger.isInitialised())
    logger.initialise();
```

6. Start the logger by calling: logger.start();
7. The data has to be allocated to the variable given by the logger before it can be logged. Afterwards, the log function has to be called, to send the data to be logged. This has to happen each cycle the data has to be logged. In the example below 100 cycles are simulated in a for loop.

```
// Setting initial values for data_to_be_logged
data_to_be_logged.this_is_a_bool = 0;
data_to_be_logged.this_is_a_int = 0;
data_to_be_logged.this_is_a_char = 'R';
data_to_be_logged.this_is_a_float = 2.0;
data_to_be_logged.this_is_a_double = 4.0;

//Cycling through 100 cycles
// And changing the data each cycle
for (size_t i = 0; i < 99; i++)
{
    // Allocating data to data_to_be_logged to be logged
    data_to_be_logged.this_is_a_bool = !data_to_be_logged.this_is_a_bool;
    data_to_be_logged.this_is_a_int++;
    if (data_to_be_logged.this_is_a_char == 'R')
        data_to_be_logged.this_is_a_char = 'A';
    else
        data_to_be_logged.this_is_a_char = 'R';
    data_to_be_logged.this_is_a_float = data_to_be_logged.this_is_a_float/2;
    data_to_be_logged.this_is_a_double = data_to_be_logged.this_is_a_double/4;

    // Sent data to the log file
    logger.log();
}
```

8. Stop the logger when it is not more needed, by calling: logger.stop;
9. After the logging work, decode the log file into a .mat or .csv file. For this, two python scripts are available in directory ros2-xenomai4-framework/XenoRosFramework/Common/XenoFrtLogger_decoder. Instructions how to use appear after calling one of the scripts, for instance, python binairy_to_csv.py

11 Software Development on Xenomai 4 / EVL

11.1 Introduction

Xenomai 4 is a framework that adds a new core to the Linux kernel called the *EVL core*, which can be used as a high-priority execution stage enabling real-time behaviour. This is a so-called "dual kernel" approach. To be able to use Xenomai 4 and its userspace utilities, the Linux kernel needs to be specifically compiled with the Xenomai 4 patches, which has been done for the RELbots.

Using Xenomai 4 implies that you can*not* compile and test code written for Xenomai 4 on your own computer, but only (remotely connected) on the Raspberry Pis of the RELbots.

This chapter provides information on how to develop, run, and debug code on the Raspberry Pi from within VS Code running on your laptop, so from there remote development on the Raspberry Pi.

How to organise your C++ code to be run in real time using EVL is next, Section 11.2. For preparing VS Code, so tuning the settings files (tasks.json, launch.json, and c_cpp_properties.json) see Section 11.3.

11.2 C++ code for Xenomai 4 / EVL

The approach is to bind a normal (Posix) thread to the EVL core, such that such a thread runs in real time using the EVL libraries. The parameters of such a thread can be specified before or directly after binding it to EVL. For this to work, the library libevl must be used, and thus added to the compile command.

It is crucial to *only* use functions from libevl, as otherwise the thread looses any real-time guarantee when a 'normal' system call is used.

Code running in real time using libevl system calls is said to run in the *out-of-band* context. Normal running code is in the *in-band* context.

A code skeleton is presented first, then subsection with links to documentation, tips, remarks, and parameters. After that, a subsection on compiler flags and configuring this in VS Code.

11.2.1 Code Skeleton

To get started with libevl, the userspace library for creating programs that use Xenomai 4, a regular C or C++ program needs to be written that uses pthread_create to spawn a new thread. This thread can then be attached to the EVL core by using the evl_attach_-family of functions.

The general structure of a thread running in real-time and doing some work, rtwork(), is in Listing 11.1.

11.2.2 Links, Remarks, Tips, and Parameters

Links to documentation on the Xenomai 4 / EVL site

- Documentation top level.
- On EVL processes, the Application Interface.
- List of all functions.
- Subpage on threads.
- Subpage on scheduling.
- Scheduling details, on oob and ib.

```
// EVL thread worker function
void* rtWork(void *arg) {
  // declarations
  // non real-time work, if any
  // set more parameters to thread, if any
  // go to out-of-band context, so make the thread an EVL thread:
  int efd = evl_attach_self("my RT worker thread");
  // check if attach has succeeded:
  if (efd < 0) {
    evl_printf("error attaching thread: %s\n", strerror(-efd));
    // handle error
  }
  // do the RT work: take care to stay out-of-band
  // go back to 'normal':
  evl_detach_self();
}

int main () {
  // declarations
  // settings, non real-time work, if any
  // set parameters of the new thread (set most parameters here)
  // let the process run on a EVL core:
  int err = pthread_create(&thread, &attributes, rtWork, NULL);
  pthread_join(thread, NULL);
  // process results of the rtWork function, if needed
}
```

Listing 11.1: Skeleton of code to be run in real time using EVL. Comments ending with a : have example code directly below it.

Links on pthreads in general

- Unix System Thread Reference.
- Blog with coding examples.
- Short text on pthreads basics
- Paper of 6 pages on threads.

Remarks, tips, and parameters

- Use the correct scheduling policy as the default is SCHED_WEAK, which is *non* real time.
- Use a proper priority, as a too low priority might let other processes go first.
- While attached to the EVL core, make sure to use *only* EVL functions, and not normal C library functions. The latter causing the thread going to in-band, so *no* guaranteed real-time behaviour anymore.
- Do *not* use memory allocation functions in the real-time part. So do *all* variable declarations outside the real-time part.
- A thread is normally pinned to the CPU core it was originally scheduled on, but you might need to explicitly tune this.
- Good practice is to use return values of functions to check whether that function did run correctly. See the use of evl_attach_self() in Listing 11.1.

Some useful evl functions

- evl_is_band to check the state of the current thread.
- sched_setaffinity to explicitly schedule the thread on the chosen CPU core. This is a C-library function, so call it outside the EVL part.

11.2.3 Compiling

To compile a program for real-time running in Xenomai, you need to make the compiler link to real-time functions instead of the normal non-real-time variants. In order to do this easily, Xenomai provides a pkg–config-file, which can output all required compiler and linker flags. The file is located in /usr/evl/lib/pkgconfig/evl.pc. In the simplest case, compiling and linking a C-file (or multiple files) in one gcc command, the required flags can be obtained as follows (try it to have an idea what is going on):

```
pkg-config /usr/evl/lib/pkgconfig/evl.pc –cflags –libs
```

Note that this only adds the flags that are required to use Xenomai headers and link against EVL, so any other required libraries (pthread, for example) need to be added by hand. One can embed the output of pkg–config directly inside a new command with the bash command substitution feature[1], for example the compile command. With that, compiling a file, say test.cpp for Xenomai from the command line becomes as easy as:

```
g++ test.cpp -o test $(pkg-config /usr/evl/lib/pkgconfig/evl.pc –cflags –libs)
```

11.3 Configuring VS Code to build Xenomai 4 code on the Raspberry Pi

This section is here for completeness and further tuning of VS Code for the building / compilation and linking process.

You do *not* need this when compiling C++ code for Xenomai 4 from the command line.

Specifying the compiler flags can be done in the tasks.json file, as arguments of the g++ command. Unfortunately, VS Code does not support shell substitution in tasks.json. To circumvent this issue, add the required compiler flags to our file. This results in the following contents:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "type": "cppbuild",
      "label": "C++: gcc build active file (Xenomai 4)",
      "command": "/usr/bin/g++",
      "args": [
        "–fdiagnostics–color=always",
        "–g",
        "${ file }",
        "–o",
        "${fileDirname}/${fileBasenameNoExtension}",
        "–I/usr/evl/include", "–L/usr/evl/lib", "–levl"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": [
        "$gcc"
      ],
      "group": "build",
      "detail": "compiler: /usr/bin/gcc"
    },
  ]
```

[1] When using $(command) in a command, the command is executed and its output is substituted there

```
}
```

launch.json (Raspberry Pi, Xenomai)

Running a real-time program in Xenomai can only be done as root, i.e., you should run the program (say, test) as sudo ./test

This can be done from within VSCode, although it seems that using breakpoints does not work out-of-the-box. A small script at /usr/bin/gdb–sudo can be created on file system of the Raspberry Pi which simply invokes the debugger gdb as root:

```
#!/bin/bash
sudo /usr/bin/gdb "$@"
```

Now in launch.json the only thing to do is to change the "miDebuggerPath" from "gdb" to "gdb–sudo".

12 Tips & Tricks and Cheat sheets

12.1 Introduction

Over the length of this document, a lot of commands are used, here we provide a collection of some often used commands. General "cheatsheets" containing more information are referenced. In line with commonly used styles, *<name>* denotes a name to be filled in by the user, with the name filled in providing a hint of what the 'type' should be. A *[ ]* will be used to describe different options if needed, with the pipe character (|) used to separate the different options.

Generic shell explainer: https://explainshell.com/

12.2 General terminal commands

A lot things will be done through the terminal, whatever system you have. Most of these commands will also work on Windows. Most terminal applications have auto-complete build in, which can be used by pressing tab. If no option can be given, a sound will play. Sometimes the application is not aware of all options: try writing them out, an error will pop up if there was a mistake, sometimes with hints or shows the string to type to get concise help info on the command itself.

Some general commands are:

- cd *[~ | ../ | <folder–name>]*: used for navigation. ~ denotes home directory in Linux and Mac, C: does the same for windows. ../ moves you back one folder. *<folder–name>* moves you into the folder named.
- mkdir *<name>*: Makes a directory called name.
- ls: List contents
- pwd: path to working directory
- cp *<location a> <location b>*: Copy file from a to b.
- mv *<location a> <location b>*: Move file from a to b.
- rm *<file>*: Remove a file.
- *<command>* ––help: Generally works to present the descriptions of options for a command. Longer explanations can be found online.
- nano *<filename>*: Show and edit file contents in the terminal, useful for checking if file matches expectations. Exit with Ctrl + X.
- Ctrl/Cmd + R: Reverse search, allows you to search back in the previous commands you have given.
- Arrow keys: Allows you to search through previous commands (up/down), or move your text cursor in a command (left/right).
- code . : Open VS Code on this location (indicated by the dot).

12.3 VS Code

General cheatsheet: Windows, Mac, Linux.

- Ctrl/Cmd + Shift + P: Opens the Command Palette, allows searching for commands. Also shows the key combination attached to a command.
- F1: Same as above

12.4 Compiler (g++ or clang)

The focus is on the command line here, not on using VS Code to build files.

Note: when compiling using VS Code's interface, be aware to select the g++ compiler, so *not* the gcc compiler, which is often the first to select from in the drop-down box of the command palette.

Cheatsheet: From GNU itself, a course given somewhere else

First, some core arguments:

- –o *<name>*: define the name of your output file.
- –c : Compile but do not link.
- –std=c++14: Set C++ Standard to C++ 14 or std=c++17 for C++ 17, obviously.
- –pthread –lpthread: Linux environments only; Makes sure that threads can be used in the program (Windows / Mac do *not* need to have this flag to properly deal with threads).

Using these arguments in combination with each other allows to write:

- g++ a.cpp b.cpp –o C: Compile a.cpp and b.cpp, and create an output called C (automagically becomes C.exe on windows)
- g++ a.cpp b.cpp –c: Only compile, but do not link. Generates output a.o b.o .
- g++ a.o b.o –o C: Take two compiled files, and link them into output file C.

12.5 WSL (Windows)

WSL does not have that many specific commands. Mostly, you use the general terminal commands for more information, see Section 12.2.

Cheatsheet: https://learn.microsoft.com/en-us/windows/wsl/basic-commands#install

- wsl: Start WSL at the given location, this is also where the WSL prompt will be.
- wsl ~: Start WSL, but go to the home of your wsl environment (\home\ubuntu)
- Inside WSL instance: explorer.exe . : opens a file explorer window at given location.

12.6 Multipass (Mac)

Multipass can be accessed through the GUI, or through a terminal / shell from the host (of course). Some Terminal commands are mentioned here. The <name> of the instance (VM running within Multipass) is "jazzy", in case you followed our install guide.

- multipass start *<name>*: start the instance (VM) with the given name.
- multipass shell *<name>*: Open a shell in the given instance,that needs to be running.
- multipass stop *<name>*: Stop multipass instance from running.
- multipass restart *<name>*: Restart multipass instance.

12.7 SSH

After setting up the SSH connection to a device, there is one main command to use:

- ssh *<user>@<ip>*: User is the username, IP is the ip address. After this, fill in the information ssh requests.
- ssh –X *<user>@<ip>*: Same as before. -X enables X11 forwarding, which lets ssh be used for showing graphics on the host produced on the VM.

12.8 ROS 2

ROS 2 is a large package, with lots of commands.

Cheatsheet: https://github.com/ubuntu-robotics/ros2_cheats_sheet/blob/master/cli/cli_cheats_sheet.pdf

- ros2 run *<package> <node name>* : Runs a specific executable (node) from the given package

- ros2 param *[get | set] <name>*: Get the value of parameter name, or set the value of parameter name.
- ros2 topic list : Give a list of all topics currently active
- ros2 topic echo *<name>* : Print the message currently being published on the topic called name
- ros2 topic type *<name>*: Give information about the type of the information on the topic
- ros2 launch *<package> <launch–file>*: Launch the executables in the launch file.

12.9 Colcon

Colcon is the build system for ROS 2. Colcon is based on CMake and Make. Colcon basically searches for all packages 'below' the location of where it is called and searches for CMakeLists.txt and package.xml to identify packages. After this, it tries to run these and build these packages, with knowledge of the ROS 2 install location for possible package-linking.

Cheatsheet: https://github.com/ubuntu-robotics/ros2_cheats_sheet/blob/master/colcon/colcon_cheats_sheet.pdf

- colcon build: Builds all package in the workspace.
  Note: Call it from workspace root, as it will place a build/, install / and log/ folder in the folder from which it is called.
- To build a specific package only (e.g. a messages package) use:
  colcon build --packages-select <message package(s)>
- *Always* set the (updated) environment variables after a successful build via: source install/setup.bash
- To exclude a folder (package) from being compiled, add an (empty) file called COLCON_IGNORE in the src/ folder of that project. Use the touch command for this in that src/ folder: touch COLCON_IGNORE Alternatively, use the --packages-ignore <package names> argument.
- In case errors are persistent even though the code was fixed, the cause might be auto-generated files being *not* updated. A solution is often to remove those:
  rm -r build/ install/ log/
- Cleaning the cmake cache or clearing generated files can be done using respectively:
  colcon build --cmake-clean-cache
  colcon build --cmake-clean-first
  It does *not* remove the auto-generated files though.

12.10 Terminal applications useful for ROS

ROS often requires multiple processes called via separate terminal windows to run at once. We present some options which we consider useful for this. Main reason for usage is the splitting of windows into *panes*, each of them having their own terminal application.

12.10.1 Linux: Terminator

Terminator is a terminal application for Linux.

- sudo apt install terminator: run once, installs the application. Run by searching for terminator.
- Ctrl + Shift + E: Split window vertically.
- Ctrl + Shift + O: Split window horizontally.
- Ctrl + (Shift +) Tab: Move to next window, or back when also pressing shift.

12.10.2 Windows Terminal

Windows Terminal should not be confused with the Command Line or Powershell applications. Installed via Microsoft Store, no account should be required. Has many options and can also run other shell types when installed.

- Alt + Shift + '+': Split window vertically.
- Alt + Shift + –: Split window vertically.
- Alt + Shift + D: Split window vertically.
- Ctrl + Shift + W: Closing Pane; if one pane, the tab closes; if one tab, the application closes.
- Alt + Arrow–keys: Move focus to other pane.
- Ctrl + Shift + P: Command Palette, allows for searching for commands

12.10.3 Mac: iTerm2

iTerm2 is a Mac terminal application which does multi-pane terminals. Install via the indicated link. Under Settings, Profiles you can set "Reuse previous session directory" to start the new terminal in the directory of the termial which has focus while starting a new terminal.

- Cmd + D: Split Window vertically.
- Cmd + Shift + D: Split Window horizontally
- Cmd + W: Close Pane; if only one pane, the window closes.
- Cmd + Option + Arrow–keys: Move between panes.

13 RELbot

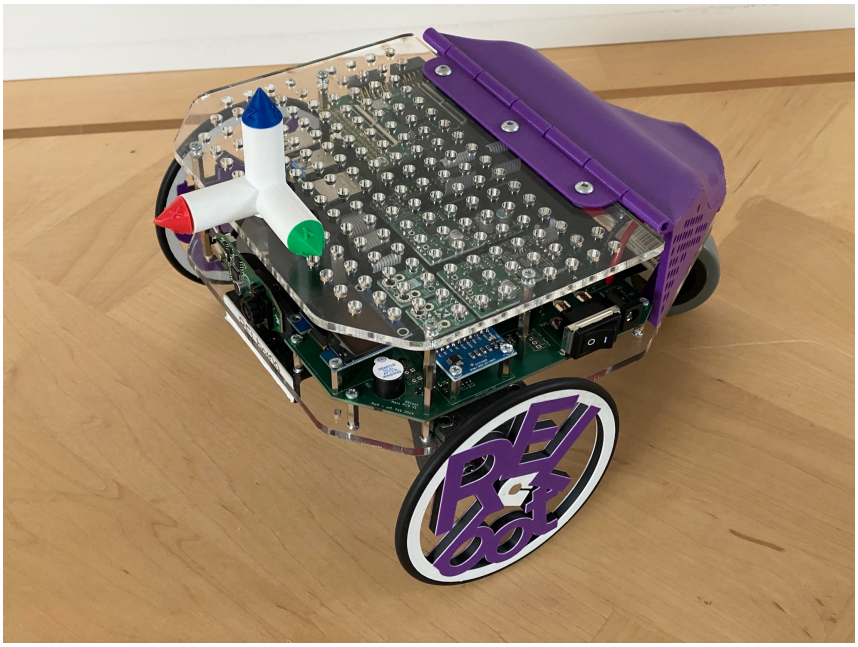

Figure 13.1: RELbot v1.1 with a coordinate frame placed on the top plate.

13.1 Introduction

The RELbot (Robotics-Education-Lab robot) is a differential-drive robot, driven by two high-quality electromotors, high-precision encoders on the motor axes, and a high-quality gearbox. See Figure 13.1 for a photo of the RELbot version 1.1, with a coordinate frame placed on top: x pointing forward, y pointing leftward, z pointing upward.

The RELbot can be considered as a rigid body that can move on a surface. The construction and layout of the wheels of the RELbot limit possible movements to driving forward or backward, or turning. For describing the *pose* (position and orientation) of the robot, we therefore only need x, y, and θ_z rotation around the vertical axis (z-axis).

In the context of current use, the surface on which the RELbot moves is considered flat and horizontal (tabletop or lab floor), simplifying the dynamic models of the RELbot needed for simulation. The RELbot itself can of course drive up or down a slope, and in those cases it behaves slightly different as predicted in simulation.

This chapter continues with an overview of the RELbot, an overview of its software, and some details on its hardware.

Chapter 14 elaborates on powering on and off, using batteries, and how to connect remotely to the RELbot.

Chapters 15 and 16 deal with the RELbot software development based on prototyping using simulation to test the software on beforehand, without the real RELbot at hand. This for both the soft real-time and the firm real-time parts, so simulation in ROS2 respectively in 20-sim. These approaches are such that it is straightforward and easy to switch between the ROS2 RELbot simulator and the real RELbot.

13.2 Overview of the RELbot

An overview of the structure is in Figure 13.2.

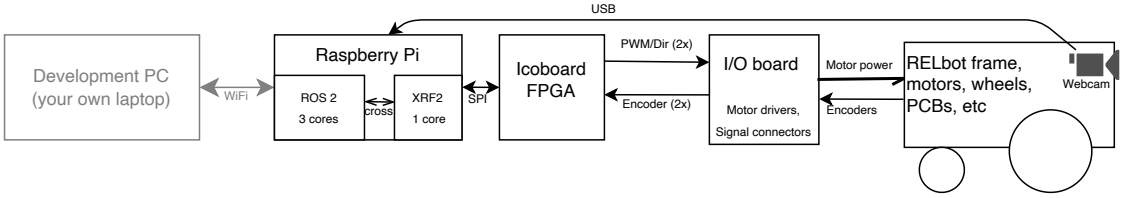

Figure 13.2: CPS structure of RELbot v1.1. Connection to a development machine is also shown.

The ROS2 nodes run on 3 cores of the Raspberry Pi 4B and the firm real-time part uses the XRF2 framework and runs on the Raspberry Pi core that has the Xenomai 4 real-time operating-system extension.

The connection between the FPGA on the Icoboard and the motor amplifiers / encoders is via PMOD connectors, one per degree of freedom. So, on one PMOD connector, both PWM and encoder signals of one motor–encoder pair are situated.

Relevant parameters of the RELbot are in Table 13.1.

Table 13.1: Relevant parameters of the RELbot.

Name	Symbol	Value	Unit
Mass	m	1.35	kg
Wheel diameter	d	0.101	m
Gear ratio	n	15.58 : 1	
Motor Constant	K_m	39	rad/s /V
Encoder		1024	counts per turn
Encoder		4	pulses per count
Supply voltage (motors)	V_{cc}	12	V

13.3 Software Overview

The RELbot software consists of three parts: *soft* real-time, *firm* real-time, and *hard* real-time[1], see Figure 13.3.

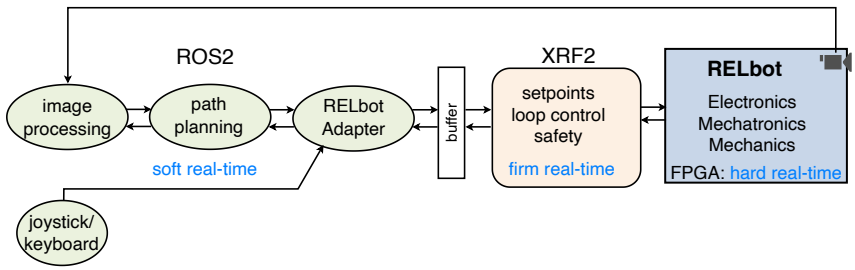

Figure 13.3: Allocation of software parts on the RELbot. It is the annotated bottom part of **??**.

- *Soft* real-time: the ROS2 nodes running on 3 cores of the Raspberry Pi. See Chapter 15 on how to develop ROS2 nodes for the RELbot, and test them using a ROS2 RELbot simulator node. Also in this chapter are supporting nodes (called *helper* nodes in (Deitel and Deitel, 2017)).
- *Firm* real-time: the 20-sim generated code (loop controllers, time-critical safety) uses the XRF2 framework and runs on the Raspberry Pi core that has the Xenomai 4 real-time

[1]Hard real-time: violating deadlines is really critical. Here, pulses must be generatated respectively counted precisely.

operating-system extension. See Chapter 16 on how to develop the application code, and test that using 20-sim simulation. Chapters 9 - 11 explain how to use XRF2 framework, its logger, and how to develop firm real-time software in general using Xenomai 4.

- *Hard* real-time: runs on the FPGA and is coded in Verilog. It runs the PWM-pulse generation and encoder-pulse counting. Data is transported to the firm-real-time Raspberry Pi core via the SPI serial data bus. In this manual, no further information on developing this part of the software is included, because it is out of scope.

13.4 Hardware overview

The RELbot has been built in a modular fashion. Literally, since as much as possible Commercial Off-The-Shelve (COTS) modules are used, e.g., motor amplifiers from Pololu.

Figure 13.4 gives an overview of the relation of all modules in the RELbot. Each module is described in the following subsections.

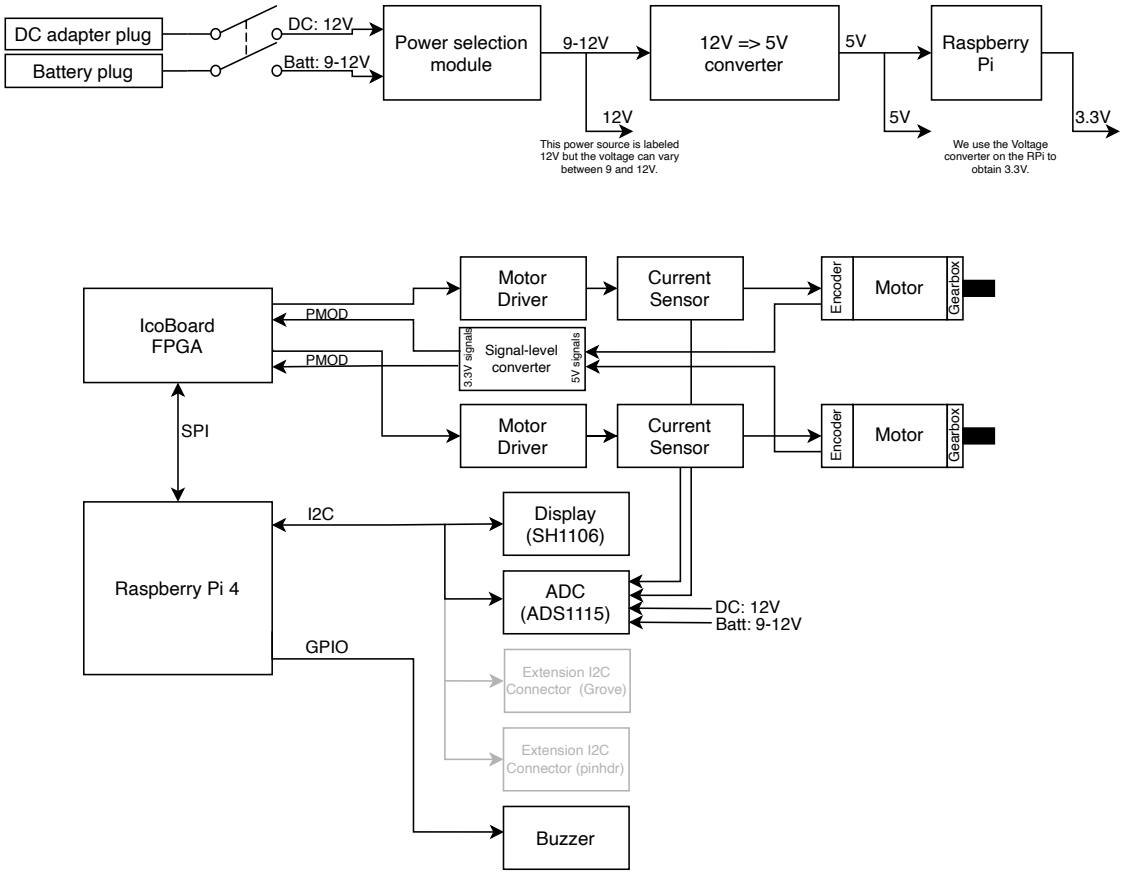

Figure 13.4: Overview of RELbot hardware. Top: power supply. Bottom: computer boards and peripherals, including motors.

13.4.1 Power-providing modules

The *Power Selection Module* takes care of 'merging' the two power sources (battery and mains). It is the only module that has been developed in-house. By design, this module prevents the battery from being charged in-system, and it allows to *hot swap* the power sources: the battery can be exchanged while the RELbot keeps 'running' provided it is connected to mains. Its functions:

- Battery under-voltage protection.
- Power source selection by two 'ideal diodes' (LTC4358).

- *Hot swappable* switching between power sources.

The *12 V to 5 V converter* board generates a stable 5 V from the supply voltage (either battery or DC adapter). Properties:

- Type: Pololu 2858, based on the D24V22F5 step down voltage regulator
- Output: 5.0 V, max 2.5 A

13.4.2 Drivetrain – Overview

The *motor driver* modules are Pololu 2999s. The *current sensor* modules are Pololu 2452s. The *signal-level converter* module is a SPI-I2C-UART Bi-Directional Logic Level Converter 4 channel from Tinytronics. This *one* module serves both encoder signals.

The *motor-gearbox-encoder* combination is the Maxon combination DC-max 26 S, B7E876B14A3D, consisting of motor DC-max 26 S, GB KL 24 V; gearbox GPX 26C, 16:1; and encoder ENX 16 EASY, 1024 IMP. Details on steering the motors and processing encoder pulses is in Sections 13.4.3 and 13.4.4.

The wheels are designed such that the 'spoke'–hub connection will break when the RELbot would inadvertently fall from some height sideways on one of its wheels. This prevents breaking less-easy replaceable parts like the motor mounts or even the base plate.

13.4.3 Steering the motors using PWM

Each motor has its own motor driver PCB, as shown in Figure 13.4. It receives PWM signal pulses and a direction[2] signal from the icoBoard FPGA.

In the control loop (firm real-time part) the PWM steering value are integer values and are processed as follows by the FPGA:

- PWM value $[-2047 \ldots 2047]$, maps to $-100\% \cdots +100\%$ of full motor power. Values outside this range are clipped to -2047 or $+2047$.
- The right motor is connected to icoBoard PMOD connector P1.
- The left motor is connected to icoBoard PMOD connector P2.
- The direction of rotation for positive/negative values is undefined, though identical for all setups. You need to try and, if needed, correct for it in software.

13.4.4 Encoder-signal processing

Each degree of freedom has an incremental encoder that is connected on the motor axis. This encoder outputs A/B pulses (*no* zero/index pulse) when rotated. It produces 4 pulses per encoder count. The corresponding signals on the PMOD connectors are called ENC_A and ENC_B.

The FPGA counts the pulses and sends this count to the Raspberry Pi (via SPI):

- The value sent is encoded as a *14 bit unsigned int*, i.e., a value in the range $[0 \ldots 16383]$. Note that this wraps around, e.g., if the encoder value is 0 and it decreases by one, the encoder value becomes 16383.
- There are 4 pulses per encoder count and the encoders have 1024 counts per revolution.
- The direction of counting is undefined, though identical for all set-ups. You need to try and, if needed, correct for it in software.
- The encoder count is *relative to the initial orientation*, i.e., the orientation the RELbot wheels were in when the Raspberry Pi was turned on. The encoder count values are reset to zero when driver program is started, so upon start of the program started by the user.

[2] Actually, two direction signals, PWM_ENA and PWM_ENB, where always PWM_ENA=!PWM_ENB.

13.4.5 IcoBboard

The icoBoard has a Lattice ICE40 Hx8K FPGA on board, with 8k logic elements. Code for it needs to be developed using Verilog. It has 4 PMOD connectors, each having 8 signal pins, power ()V_{CC}) and ground pins. More connectors are available, but are not used for the RELbot and do not have code for it in the Verilog program currently in use.

The FPGA is connected via the SPI serial bus to the Raspberry Pi. The Verilog program on the FPGA generates PWM pulses from the steering-value percentages (see Section 13.4.3), and counts the encoder pulses into an unsigned integer value (see Section 13.4.4).

The connection between the FPGA on the Icoboard and the motor amplifiers / encoders is via PMOD connectors, one per degree of freedom. So, on one PMOD connector, both PWM and encoder signals of one motor–encoder pair are situated.

13.4.6 Supporting modules

The *ADC* module is the ADS1115 16-bit ADC I2C - 4 channel, converting voltages of the power sources and currents of the motors to digital signals.

The *Display* is a 1.3 inch OLED Display 128*64 pixels White, I2C. It shows the date (American format) and time; DC voltage (from charger); battery voltage; the wired Ethernet address (`eth0`) and the wireless Ethernet address (`wlan0`).

Note: the RELbot only listens to a few specific WiFi networks. For sure, *not* to the standard `eduroam` network.

The *base PCB* and GPIO breakout PCB (to get GPIO signals both on the icoBoard and base PCB) have been developed in-house, as these are rather specific.

14 RELbot Start up, Shut down, Battery, Store in Box

14.1 Quick Guide

See for details the sections after this list.

- Power: use either the supplied 12 V power supply or supplied 12 V LiPo battery.
- *Hot swappable* power sources: switch to the other power source while the RELbot is running its software.
- Testing: keep the RELbot on its stand.
- Let it ride: before let it ride on the ground (or table), test it first while on its stand.
- Shutting down:
  - *First* stop the software via the command: `sudo shutdown now`
  - *Then* wait until OS is down, and *after* that, power off the RELbot.
- Return after use: put all items in the box:
  - All with the *same* RELbot number on it: 12 V power supply, RELbot flag, and the RELbot itself.
  - Without a RELbot number on it: green ball, stand, Quick User Guide.
- Battery: return separately, so *not* in the box, as it needs different way of storing. The battery does *not* need to have the same REL*bat* number on it as the RELbot itself.

14.2 Start up

After connecting the DC adapter or battery, flip the switch on the side of the RELbot. A green LED on the main PCB, close to the switch, and the power LEDs on the Raspberry Pi turn on.

The RELbot's Raspberry Pi needs some time to boot after the RELbot has been powered on. After this boot procedure, status information is shown on the display:

- Supply voltages.
- IPv4 addresses of cabled and WiFi connections, if connected.
- Current wall-clock time.

In case the RELbot does not start up properly, check all supply voltages:

- 12 V from power supply / battery.
- 12 V (at least 11 V), 5 V, and 3.3 V on the main PCB, using their test points on the main PCB, located next to the battery connection wires.

14.3 Using the RELbot

About using the RELbot is in other chapters and sections of this RELbot manual, most notably:

- Set up an SSH connection to the RELbot's Raspberry Pi, see Section 4.4.2.
- Show graphics produced on the RELbot on you own laptop, see Section 4.8.
- Access the Raspberry Pi of the RELbot via SSH, see Chapter 8.
- On the RELbot simulator for developing SRT software, see Section 15.2.
- Run soft real-time software using ROS2, see Chapter 15.
- About the XRF2 framework to run the generated firm real-time software in, see Chapter 9.
- On the RELbot simulator for developing FRT software using 20-sim, see Section 16.2.
- Run firm real-time software generated with 20-sim see Chapter 16.

14.4 Shut down, store in box

- First, properly shut down the Raspberry Pi via the command on the Raspberry Pi: `sudo shutdown now`

- Wait a while for the Raspberry Pi to complete the shutdown process. Failure to do so may corrupt your SD card (i.e., you lose all your files).
- Turn the robot off with the switch on the side.
- Disconnect the DC adapter.
- Disconnect and remove the battery from the robot.
- Store the robot, stand, DC adapter, and other accessories in the box.
- Store the battery separately, as it has to go to a safe place (e.g., a designated battery storage closet).

14.5 Background on batteries and power supply

DC adapter operation

- Required adapter: *12 V, 5 A*.
- Required plug: DC barrel jack, 6.3 / 2.1 mm (center-positive)

Battery operation

- The battery used is *3S LiPo*, 11.1V, 2200 mAh. Connector: XT60. Batteries are labeled 'RELbat' and are numbered. However, batteries are *not* one-to-one related to a specific robot; you can use any RELbat in any RELbot.
- When charging *do not charge beyond storage voltage (11.4 V)*. The reason for this is that a battery should not be above storage voltage for a prolonged time. By never exceeding the storage voltage it is not necessary to keep track of this, saving a lot of administrative work. Of course, this reduces the effective capacity of the battery. This fact was taken into account when selecting the capacity of the batteries.
- There is a hardware battery-under-voltage protection circuit which makes sure that the battery can never be fully drained. When the battery goes below *9.0 V* the battery power is switched off (minimum turn-on battery voltage is 9.8 V). Note that there is *no* under-voltage protection on the DC-input.
- The battery circuit is *protected with a 12/24A poly-fuse* (automatically resetting fuse which trips somewhere between 12 and 24 A). This fuse is quite slow (takes a few seconds to trip, depending on the actual current); its sole purpose is to prevent a continuous direct shortcut.
- If both the battery and DC adapter are connected, the battery is *not* charged. Instead, use a stand-alone charger to charge the battery outside the robot.
- A battery lasts for at least one hour of continuous operation; usually (much) longer. Depends on how much the robot drives.

Hot swappable

The RELbot's power circuit is hot-swappable, i.e., you can switch between DC adapter and battery operation while the robot is on. To do this, simply connect the second power source before disconnecting the first.

As long as both power sources are connected, the power source delivering the highest voltage will power the full system. This implies that, even if there is a DC power adapter connected, the system may be battery-fed. This is by design and has the following advantages:

- This will automatically (partly) drain the battery if someone has charged it beyond storage voltage (see above).
- The battery serves as a backup for the DC adapter; if there is a high-current surge beyond the capacity of the DC adapter (causing the DC adapter voltage to drop drastically), the battery, which can deliver a much higher current, will take over (usually for much less than a second). In practice however, we have never experienced a situation where the DC adapter voltage dropped such that it gave problems.

15 Developing Soft Real-Time Software for RELbot using ROS2

15.1 Introduction

The *soft* real-time software controls the *firm* real-time software and thus the mechatronics of the RELbot, reacting on signals it receives from those parts. Therefore, for testing the soft real-time software, it is relevant to simulate the 'missing' part (that is, the rest of the system, so all *non* firm real-time parts). Furthermore, carefully structuring the software and the simulator is important and must be such that switching from simulation to the real RELbot is only changing the simulator node by the connection to the real robot.

The provided RELbot simulator node and the RELbot Adapter node are designed such that indeed switching from simulated RELbot to the real RELbot is simple and straightforward: only changing a parameter in the RELbot Adapter to send its outputs (the steering values to the firm real-time part) to the right part.

Figure 15.1 shows the ROS2 nodes for the RELbot; it is a more detailed diagram compared to the corresponding diagram (left middle and bottom) of **??**. Nodes in red are an example of nodes that must be developed by the user (these nodes are *not* provided) and are the *System under Test*, SUT. The green nodes are provided by the Robotics Education Lab, and are explained in this chapter. The white node is from the ROS community, providing a visualisation of the robot pose.

Switching between simulation and real RELbot is by changing the RELbot Adapter parameter robotmode between "sim" and "real", see Section 15.4. During simulation, the laptop webcam must be used, because the real RELbot and its webcam are not available nor used. While using the real RELbot, the webcam is the real webcam mounted at the front of the RELbot.

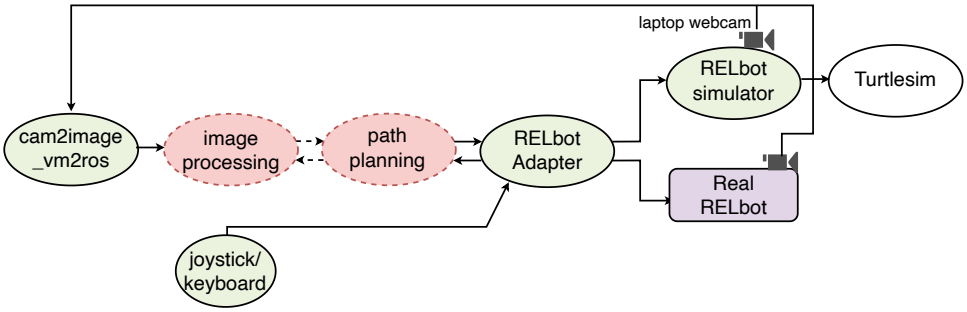

Figure 15.1: Node set up for testing SRT software and running ROS2 nodes on the real RELbot. Switching between simulation and real is by changing parameter robotmode in RELbot Adapter, such that either the RELbot simulator or the Real RELbot is connected. keyboard or joystick can be used for initial tests or extra control, or left out.

The provided nodes are designed such that for migrating to real RELbot, none of the own-developed nodes need to be changed. Of course only, when these are designed and tested very carefully.

The provided nodes and references to further explanation are:

- Cam2image adapted from Cam2image node provided by the ROS community. Adaptation is needed to use it in simulation on a VM, so the camera stream must be transported from host computer to the virtual machine running Linux and ROS2. Doing so, guarantees the same interface at the ROS2 side as the original Cam2image node, contributing

to the switch from simulation to real being straightforward. See Chapter 7 for details and how to use it.

- joystick/keyboard two nodes providing joystick respectively keyboard input. These are slightly adapted from corresponding nodes of the ROS community. The joystick node does not yet run, as it needs a kernel update. See Section 15.5 for details and how to use the keyboard node.
- RELbot Adapter connects the ROS2 nodes to either the simulator or to the real RELbot. It furthermore does safety checks, by limiting extreme steering commands. It *must* therefore *always* be used in combination with the RELbot Simulator and/or the real RELbot. See Section 15.4 for details, how to use it, and how to tune its parameters.
- RELbot Simulator contains the simulation of both the dynamic behaviour of the RELbot and the effect the moving RELbot has on what part the simulated camera sees from the video stream coming from Cam2image. It is the 'missing' part seen from the point of view of the soft real-time software (ROS2 nodes). See Section 15.2 for details, how to use it, and how to tune its parameters.
- Turtlesim takes care of visualisation of the position and orientation (*pose*) of the simulated RELbot. It is indeed the standard Turtlesim node as provided by the ROS community.

How the pose of the RELbot and thus its camera influences the image it views is indicated in Section 15.3.

15.2 RELbot_Simulator ROS2 Node

15.2.1 Functionality

The relbot_simulator package provides a model of the RELbot and a loop controller as a ROS2 node for testing ROS2 nodes, so SRT software. The model of the dynamics of the mechatronics and the loop controller are generated by 20-sim.

Furthermore, the node processes the image stream from a webcam to show the effect of the RELbot simulated movement on what the RELbot webcam would see, by outputting a part of the inputted webcam stream.

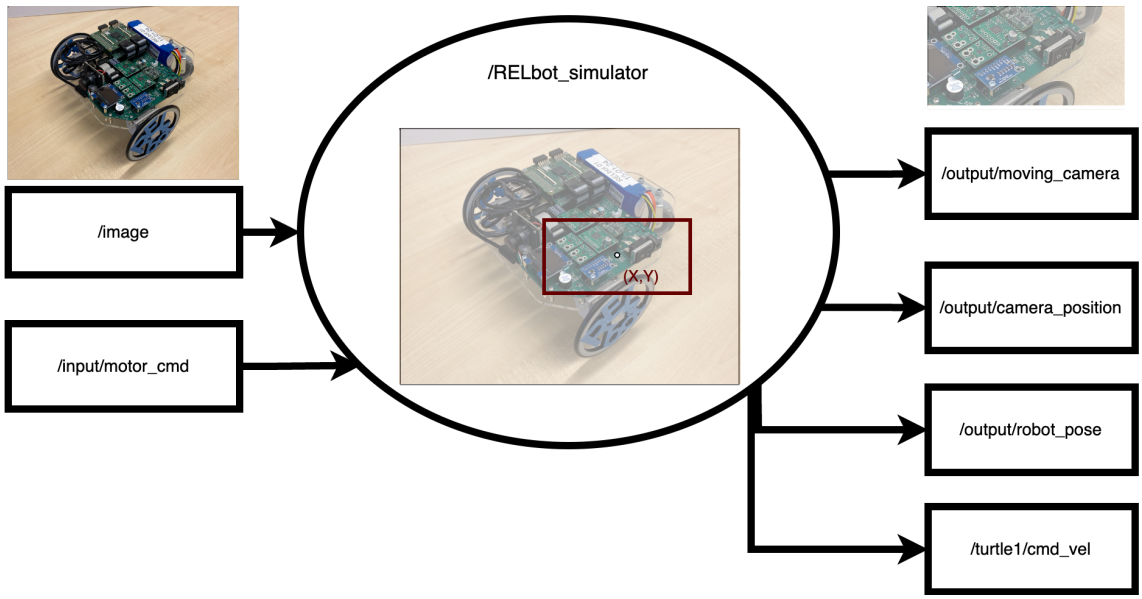

Figure 15.2: Diagram of the interfacing of the RELbot Simulator node.

15.2.2 Building

Install the package as usual, so put the folder relbot_simulator with all its files and subdirectories in the src folder of your workspace (see Section 6.2 on structuring ROS2 files).

In general, the normal build and configure command can be used.

However, in case the OpenCV ROS package has not been installed (see Section 4.3), install that first via:
sudo apt install ros-humble-cv-bridge

Note: First build custom message types, re-source your local workspace, and then you can use the new messages in packages, see Section 6.4.

15.2.3 Interfacing

Parameters

The parameters which can be used to change some behaviour of the node:

- image_stream_FPS: Controls how often the image-processing functionality is run. Default = 30 FPS.
- throttle_rate: Rate at which to throttle messages. Waits for the specified amount of time before allowing a new message to be printed. Type: double, default: 5000 ms
- visualisation_frequency_turtlesim: Sets how often the calculation towards turtlesim should be triggered. Type: double, default: 62.5 Hz.

Inputs

- /image of type sensor_msgs/msg/image
- /input/motor_cmd of type example_interfaces/msg/RelbotMotors.
  It is a set of two floats, consisting of left_wheel_vel and right_wheel_vel

Outputs

- /output/moving_camera of type sensor_msgs/msg/Image
  It is a part of the original input image, scaled and rotated to show the effect of the RELbot movement on what its camera would see. It depends on the X and θ_Z of the RELbot's pose. Default size of the image is 0.5 * height of the original image, keeping the aspect ratio. The centre of the output image is looking at /output/cam_position. Publishing rate is image_stream_FPS with default of 30 FPS.
- /output/camera_position of type geometry_msgs/msg/PointStamped
  Only X and Y components are used. Publishing rate is image_stream_FPS with default of 30 FPS.
- /output/robot_pose of type geometry_msgs/msg/PoseStamped
  It is the location of the RELbot in three-dimensional space. Only X, Y and Rotation_Z are used, describing X, Y, θ_Z respectively. Publishing rate is 1000 Hz, the timing of the dynamics of the RELbot mechatronics.
- /turtle1/cmd_vel of type geometry_msgs/msg/Twist
  Output to send to a turtlesim window. The first turtle will be controlled by this command. Publishing rate is visualisation_frequency_turtlesim with default 62.5 Hz.

15.2.4 Running the node

- To tun the node with default behaviour use: ros2 run relbot_simulator relbot_simulator
- There is no auto-stop on the simulation, so commands keep on being applied even if the image stops moving.
- Parameters can be added to the normal launch command by writing:
  −−ros−args −p *<parameter name>*:=*<value>*

Troubleshooting the RELbot simulator node

- After downloading, make sure you first build the node, after which you source the setup file again: source install /setup.bash
- Make sure an image stream is being sent to the /image topic. Without this the simulator will not show an output image. The model will continue to update.
- Encountering a GTK or QT error? Try restarting the terminal shell you are in. Make sure the ssh argument –X is used!

15.2.5 Differences with a real RELbot

There are some differences between the simulated and real RELbot. The essential ones are:

- The PWM signals, encoder pulses, power amplifiers, and signal converters are considered ideal, so their effects besides passing through the signals, are neglected.
- The simulated RELbot has some limits, outside of which the behaviour is non-crashing but the output image will generally not work:

	Simulator	Physical RELbot	Unit
x position	± 5	$\pm\infty$	m
θ_Z	$\approx \pm 2$	$\pm\infty$	rad

15.3 From coordinates to image

Figure 15.3 shows two components that affect /output/moving_image. The pose is defined from the viewpoint of the camera, so a positive rotation means the RELbot shows more towards your right, but the camera image moves more towards the left. So, a positive rotation causes the *moving_image* portion moving to the right, and thus moving a fixed point (e.g. centre) of the image to the left.

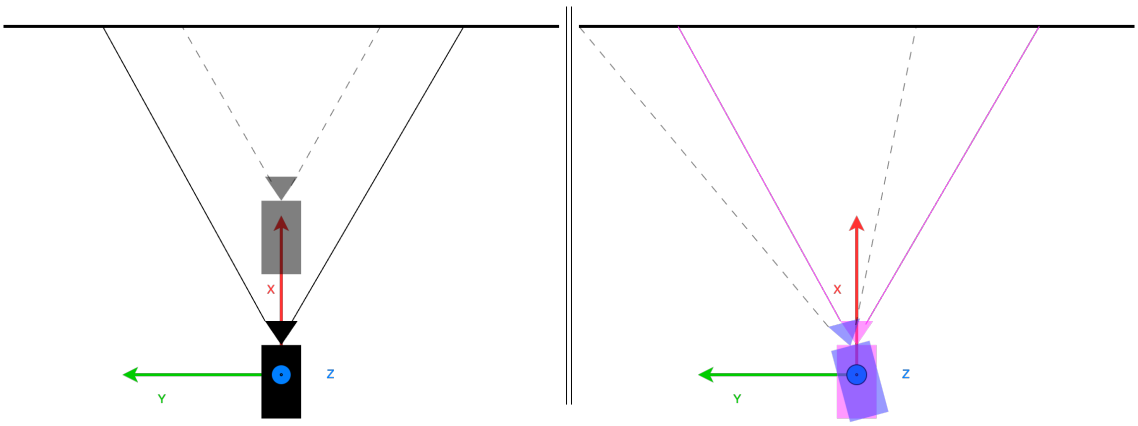

Figure 15.3: Top view of how input affects the output image. Left: Increasing x implies zooming; Right: Positive rotation (from blue camera position to the pink one) shifts centre of what the cut-out camera sees, to the left

15.4 RELbot_adapter node

Important: The relbot_adapter node must ***always*** be used, as it checks on validity of the steering signals, just like a set() method in C++ programming.

15.4.1 Functionality

The relbot_adapter package implements a standardised input interface for both the *simulated* RELbot and the *real* RELbot.

It checks if communication with the real RELbot is possible and selected. It defaults to communicating to the simulated RELbot. In this way, users do not need to change their own nodes, but only need to change the robotmode parameter given to this node.

The relbot_adapter limits incoming steering values to within given limits. Default limits are 0.25 m/s or 5 rad/s. Limits can be specified in m/s or rad/s (for the wheels) and the lowest of the two is adopted, keeping into account the relation between these two, obviously. As such, it functions as a set() method used in C++ programming.

15.4.2 Building

Install the package as usual, so put the folder relbot_adapter with all its files and subdirectories in the src folder for your workspace (see Section 6.2 on structuring ROS2 files).

In case of using for simulation only, the normal build command must be used.

In case of connecting to the real RELbot, the XRF2 must be added first, by building its message package:

- Build the xrf2_msgs package, see Section 9.2, the *Messages* part.
- Run colcon build –make-clean-cache to detect the change.
- Run a normal build (colcon build). In this complete build, the code should auto-detect that XRF2 can be used.

To remove XRF2 support:

- Delete xrf2_msgs in the workspace install folder.
- Run colcon build as usual.

15.4.3 Interfacing

Parameters

These parameters are to control the input and output of the system. This is combined with internal checks to create valid output, given the environment.

- max_speed_mps: Max speed in meters per second. Type: double, default: 0.25 ms.
- max_speed_rads: Max speed in radians per second. Type: double, default: 5.0 rad/s.
  - Both max speeds are evaluated together. Lowest will be applied. See status message on startup.
- robotmode: either "real" or "sim". Type: string, default: "sim".
  - Using "real" without availability of XRF2 will update this value to "sim".
- use_twist_cmd: Use twist as input. Type: bool, default: false.
- throttle_rate: Interval between outputting messages in milliseconds. Type: double, default: 500 ms. First message after this time period is passed get published again, after which the wait period starts again.

Inputs

These topics are used to set the desired velocity of the wheels in radians/second. The two linear velocities are mutually exclusive with the twist topic, so use either the velocity inputs or the twist inputs.

- /input/left_motor/setpoint_vel of type example_interfaces/msg/Float64.
- /input/right_motor/setpoint_vel of type example_interfaces/msg/Float64.
- /input/twist of type geometry_msgs/msg/Twist.
  - *Only* if parameter use_twist_cmd is true.
  - **Note:** Twists are expressed in SI units. Linear components therefore have a large effect as the wheels of the RELbot are rather small.

Tip: If you are unaware what a twist is, or do not know how to use it, use the wheel velocities.

Outputs

- /output/motor_cmd of type relbot_msgs/msg/RelbotMotors.
  - Default output topic, to be connected to the RELbot_simulator package.
- /Ros2Xeno of type xrf2_msgs/msg/Ros2Xeno.
  - For the real RELbot.

15.4.4 Running the node

Figure 15.4 shows the interfaces of the RELbot_adapter node and its connections to the simulator and real RELbot.

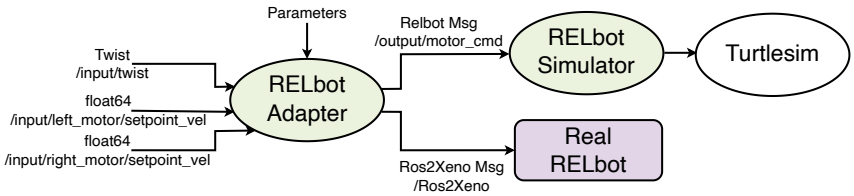

Figure 15.4: RELbot_adapter node with connections to the simulator and to the real RELbot.

Tips and Remarks

- To run the node with default behaviour use: ros2 run relbot_adapter relbot_adapter
- To run the node with all parameters set from the terminal connecting to the real RELbot, type all on *one* line and *two* dashes before the ros-args argument:
  ros2 run relbot_adapter relbot_adapter - -ros-args -p use_twist_cmd:=true -p max_speed_mps:=0.5 -p max_speed_rads:=10.123456789 -p robotmode:="real"
- Putting such a command in a launch file will save repeatedly typing such a long command.
- The /ROS2Xeno messages are non-standard messages, and need to be built first, see Section 9.2, *Messages* part. This is *only* needed when connecting to a real RELbot.

15.5 Keyboard ROS2 Node

15.5.1 Functionality

The RELbot_Keyboard package implements a node that reads from keyboard and sends a standard *twist* command to be used in, for instance, the RELbot adapter node. This package is adapted from the standard turtlebot_teleop_key node. Next to a different output format, only the arrow keys, F (stop moving) and Q (stop the node) have been implemented.

15.5.2 Building

Install the package as usual, so put the folder relbot_keyboard with all its files and subdirectories in the src folder of your workspace (see Section 6.2 on structuring ROS2 files).

The normal build and configure commands can be used.

15.5.3 Interfacing

This node has only outputs in the ROS2 sense, and no ROS2 inputs nor parameters. 'Inputs' are from reading the keyboard.

Inputs

Only these turtlebot_teleop_key keyboard commands:

- Keyboard arrow keys to produce a twist to let the RELbot animated glyph move.
- Keyboard keys F and Q to stop the current movement or quit the program, respectively.

These commands are shown when the node is started.

Outputs

- /keyboard/Twist of type geometry_msgs/msg/Twist

15.5.4 Running the node

In a separate terminal run: ros2 run relbot_keyboard relbot_teleop_key

Note: This node must *always* be run from within a terminal. From this terminal *only*, the keyboard commands will reach the node. The focus of your keyboard must be on that terminal to get the keyboard input arriving at the keyboard node. This is the reason why this keyboard node can *not* be specified in a launch file.

Tips and Remarks

- *Always* and *only* run this node from a terminal.
- Have the keyboard focus on the terminal from which this node has been started.

16 Developing Firm Real-Time Software for the RELbot using 20-sim and XRF2

16.1 Introduction

The *firm* real-time software realises the control laws, firm real-time safety, and drivers to/from the mechatronics parts, in order to control the mechatronics of the RELbot, reacting on signals it receives from those parts. Therefore, for testing the firm real-time software, simulating the 'missing' (mechatronics) part is relevant.

The control laws to be implemented and the mechatronics (its dynamics and kinematics) to be simulated have a different MoC[1] from software, and therefore need tooling that is different from tools for software development. We use the tool 20-sim for this such that thorough testing using 20-sim and using its code-generation feature is exploited to have an elegant workflow.

The provided 20-sim models for a basic RELbot control law and the RELbot mechatronics are structured such that the code generated from it can easily be used in the XRF2 framework. It is not as direct as switching between simulated and real RELbot as is done in the soft real-time software development, but it does provide a *model-based* development approach[2].

Figure 16.1 shows the structures for simulating the firm real-time part and code generation of it fitting that into the XRF2 framework, thus using the existing software infrastructure on the RELbot. Tests on the real RELbot can be the same as on the simulated RELbot. The figure is a more detailed diagram compared to the corresponding diagram (right middle and bottom) of **??**.

Going from simulation to the real RELbot is by generating code for the firm real-time loop and putting that in the XRF2 framework. The test signals go into a ROS2 node to control the test experiments using the real RELbot including the XRF2 framework, so the situation as used in the end. Use *always* the RELbot Adapter node, as that node does checks on validity of the steering signals.

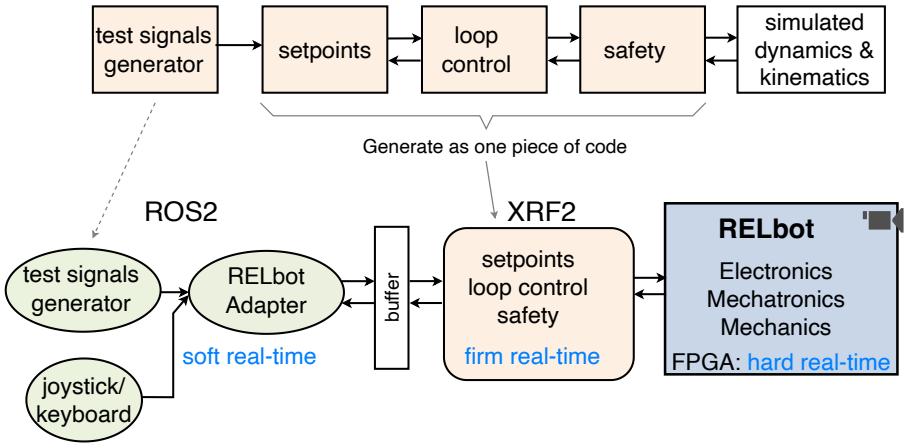

Figure 16.1: Submodels for testing FRT software in 20-sim and converting to FRT software running on the real RELbot. The test signals generator block goes to a ROS2 node to be able to test drive the real RELbot. keyboard or joystick can be used for initial tests or extra control, or can be left out.

[1] Model of Computation (see **??**). The dynamics of the RELbot must be computed precisely, far more precise than for the soft real-time parts.

[2] Switching between simulated and real RELbot for firm real-time development needs specific electrical switching and computing equipment that is *not* available. This way of working is called *Hardware-in-the-Loop Simulation*, and has been prototyped using the RELbot by Hooglander (2025).

The 20-sim model of the complete but simplified controlled RELbot is available, and is explained in the next section.

16.2 RELbot Dynamic Model in 20-sim

The complete simplified 20-sim model of RELbot includes both the mechatronic part (*plant*) and a loop controller, see Figure 16.2.

This model is available on Canvas.

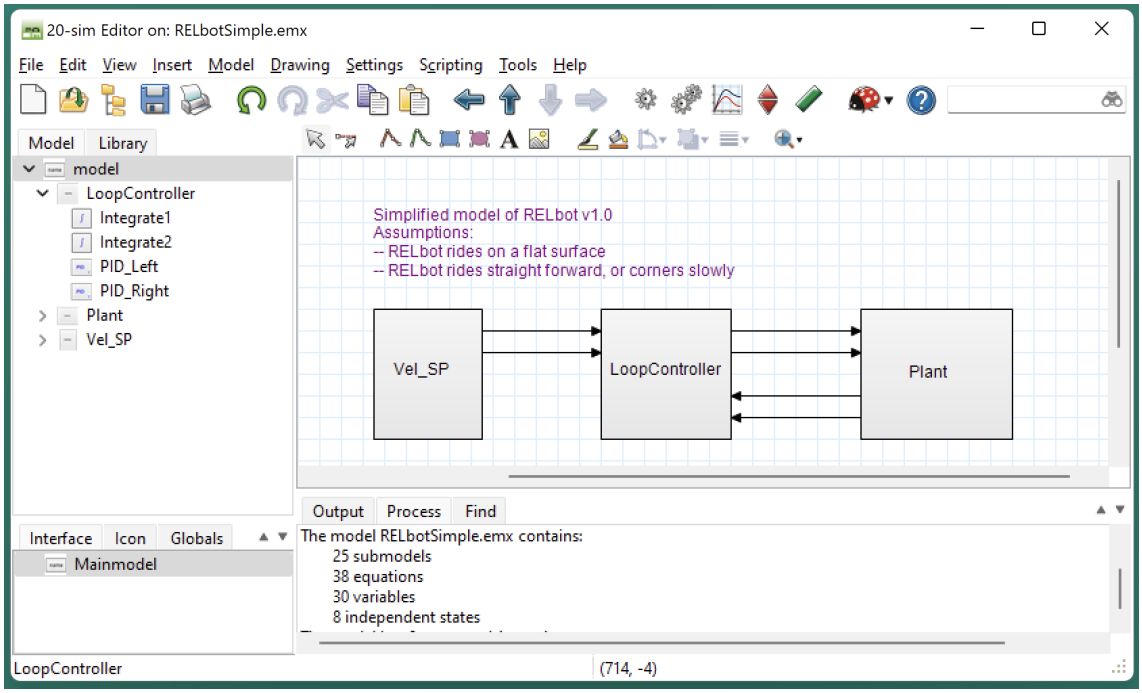

Figure 16.2: Top-level model of RELbot, shown in 20-sim's Model Editor.

16.2.1 Mechatronics Part ()Plant

The mechatronics part is called Plant in Figure 16.2.

Assumptions

- The RELbot rides on a flat (horizontal) surface.
- The RELbot rides straight forward and backwards, or takes corners rather slowly.
- Connection from / to the controller is in percentages / radians, so not actual signals.

The first two assumptions allow to model the mechatronics part as two separate dynamic parts, each consisting of a moving mass. The third assumption is the controllaw-design point of view, i.e. step 2 in the method.

Interfacing

- SteeringLeft, SteeringRight: input steering values in percentages for the left and right motor.
- EncoderLeft, EncoderRight: output values of the left and right encoders in radians mounted on the motor axis of the left and right motor, respectively.

16.2.2 Loop Controller

The Loop Controller itself is shown in Figure 16.3.

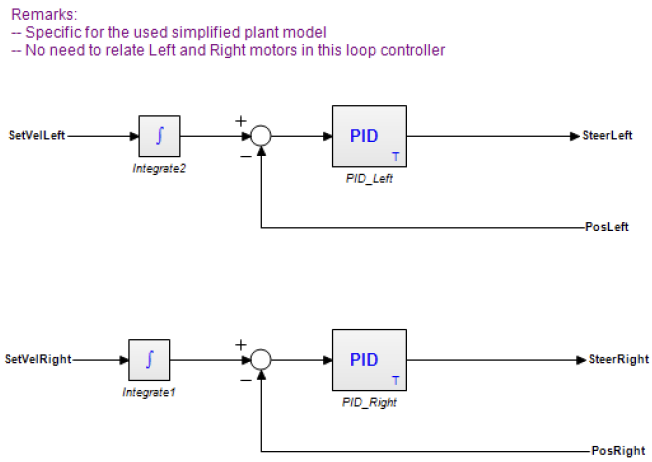

Figure 16.3: The contents of the ControllerTilt block from Figure 16.2.

Assumptions and Description

The setpoint (SetValLeft / Right) is integrated to an angle (rotational position), as the plant delivers encoder values, being angles (rotational position).

The connection to the mechatronics part (Plant) is considered ideal, so no conversions between parts to go to actual signals. This is the controllaw-design point of view, i.e. step 2 in the method.

Interfacing

- SetValLeft, SetValRight: input setpoints to the control law, as velocity in rad/s.
- PosLeft, PosRight: input measured values, in radians.
- SteerLeft, SteerRight: output steering values, in percentages.

16.2.3 Setpoint Generator

The *VEL_SP* block produces velocity setpoints for the LoopController, and serves as a test-signals generator.

This block can also be interpreted as a sequence controller.

16.3 Prepare 20-simsubmodels for use in XRF2

Use the following instructions to generate code from a 20-sim model suitable for use in XRF2. Some background information is given in Section 16.4

- Use LoopController as *name* of the submodel to generate code from.
- In the 20-sim Simulator start the C-Code Generation toolbox, via the button "C-Code Generation" (or menu entry Tools | Real Time Toolbox).
  The Simulator can be started from the button "Start Simulator" or menu entry Model | Start Simulator.
- In the C-Code Generation toolbox
  - Use the *Target* C++ class for 20-sim submodel.
  - Select the submodel (should be named LoopController).
  - Specify the directory where the code is put into.
  - Generate the code via the OK button.
- Put the generated code into the directory controller of the 20-simtemplate code, as explained in Section 9.2, "Incorporate the code..." part.

16.4 Background information on code generation using 20-sim

20-sim (Controllab Products, 2025) is a modelling and simulation package by Controllab Products (spin-off company of the UT). It consists of a model editor, a simulator, a code generator, and much more.

16.4.1 Installation

20-sim runs only on Windows. The current version can be downloaded from https:/www.20sim.com/downloadnext/ skipping filling in your personal data. The license key is avaliable on Canvas.

For Mac users, you would need a virtual machine running Windows. Virtual Box runs only on X86-based Macs and for ARM-based Macs, you can use UTM as virtual machine engine. Parallels is another good virtualisation tool for all kinds of Macs, but needs license costs.

16.4.2 Code Generation

Code generation is available in the simulator, not in the editor. This is because the code generator needs a checked-and-compiled model and that is produced when invoking the simulator. The code generation function can be reached via the button "C-Code Generation" (or menu entry Tools | Real Time Toolbox).

For use in XRF2, you need to generate a *C++ class for 20-sim submodel* using that 'target'.

You can only generate code of individual submodels; if you want to generate a single C++ class from multiple submodels then you need to group the submodels first, using for instance the function Implode, available in the context menu when selecting submodels.

The default location where the generated files are saved is C:\temp.

In case none of your group succeeds in running 20-sim you can request the generated code, and use that, instead of checking the model and generating code yourself.

A Change Log

- **Version 2025.1**, 6-2-2026
  - First version of the RELbot Software and Usage Manual.
  - **Note** Minor updates might be published later.

Bibliography

Controllab Products (Jan. 2025). *20-sim*. URL: http://www.20sim.com/.

Deitel and Deitel (2017). *C++ How to Program*. 10th. Prentice Hall. ISBN: 978-1-292-15334-6, or 978-0-134-44823-7.

Hooglander, F.J.P. (Mar. 2025). *Development and Testing of a Hardware-in-the-Loop Simulation Infrastructure*. MSc Thesis 014RaM2025. University of Twente. URL: https://cloud.ram.eemcs.utwente.nl/index.php/s/r3YtWQePwkW6wim.

Raoudi, I. (2024). *ROS2 – Xenomai4 Real-time Framework on Raspberry Pi*. MSc Thesis 078RaM2024. University of Twente. URL: https://cloud.ram.eemcs.utwente.nl/index.php/s/g6SijsJ4P8igszo.